PROJECT REPORT

Cryptanalysis of Weak Password Hashing Systems and AI-Based Defence Mechanism

Subject:	Information Security
Submitted To:	Dr. Mukhtiar Bano
Submitted By:	Amina Noor (2023-BSE-007) Hamail Fatima (2023-BSE-023) Hajra Sarwar (2023-BSE-022)
Semester:	6th (A)

MAY 28, 2026

FATIMA JINNAH WOMEN UNIVERSITY, RAWALPINDI
Software Engineering Department

ABSTRACT

Passwords remain the primary authentication mechanism in modern digital systems, yet a significant portion of users continue to rely on weak, predictable credentials that are trivially compromised. Legacy hashing algorithms such as MD5 and SHA-1, widely deployed in the 1990s and early 2000s, were designed for speed rather than password security. Their computational efficiency makes them particularly vulnerable to high-speed dictionary and brute-force attacks.

This project — Cryptanalysis of Weak Password Hashing Systems and AI-Based Defence Mechanism — investigates these vulnerabilities through a rigorous academic and practical methodology. The system generates a dataset of commonly used weak passwords, computes their MD5 and SHA-1 hashes, and subjects them to both dictionary and brute-force attacks using an interactive Python-based Streamlit application. Experimental results demonstrate a 100% attack success rate on the dataset within under 10 seconds, exposing the critical inadequacy of legacy hashing for password storage.

In response, the project implements a multi-layer defence architecture comprising bcrypt and Argon2 memory-hard hashing with built-in salting, a rule-based password policy enforcement engine, and a Random Forest machine learning classifier trained on 21 extracted password features to automatically identify and flag weak credentials. Security improvements are quantified through timing benchmarks, success-rate comparisons, and entropy analysis. The entire system is built with Python 3.12, visualised through Matplotlib, and exposed as an interactive Streamlit web application developed in Visual Studio Code with version control managed via GitHub.

Streamlit web application:
https://ai-hash-crack-defense-z6pcs5zxivvtjwum7e95zb.streamlit.app/

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION

1.1 Background

In the field of information security, password-based authentication remains the most widely deployed method of identity verification. Despite advances in biometrics and multi-factor authentication, the simple text password continues to guard the overwhelming majority of digital accounts worldwide. The strength of this defence, however, is critically dependent on two factors: the complexity of the password chosen by the user and the method by which that password is stored on the server.

Historically, web applications stored user passwords by computing a fast cryptographic hash and storing the result. Algorithms such as MD5 (Message Digest 5), introduced in 1991, and SHA-1 (Secure Hash Algorithm 1), standardised in 1995, were adopted widely for this purpose. Both algorithms were designed to be computationally fast — a property that is desirable for data integrity checking but catastrophic for password storage. On modern hardware, an attacker equipped with a single GPU can compute billions of MD5 hashes per second, reducing the problem of cracking a weak password to a matter of milliseconds.

Compounding this problem is the prevalence of weak user passwords. Studies of leaked credential datasets consistently reveal that the most common passwords — '123456', 'password', 'admin', 'qwerty' — appear millions of times. When these passwords are protected only by MD5 or SHA-1 without salting, a single pre-computed rainbow table lookup instantly recovers the plaintext. This project addresses both vulnerabilities: the algorithmic weakness of legacy hashing and the behavioural weakness of predictable passwords.

1.2 Problem Statement

Despite the well-documented weaknesses of MD5 and SHA-1, many legacy systems continue to use these algorithms for password storage, exposing millions of users to credential compromise. Simultaneously, end users consistently select low-entropy passwords that are trivially enumerated by dictionary word lists. There is a need for a comprehensive academic demonstration that quantifies these vulnerabilities through empirical attack simulation, and proposes and implements a measurably more secure alternative architecture.

1.3 Purpose of the Project

This project is designed to: (1) demonstrate the practical exploitability of MD5 and SHA-1 password hashes through dictionary and brute-force attack simulations on a realistic weak password dataset; (2) analyse the results using entropy metrics, statistical visualisation, and machine learning; and (3) implement and validate a defence architecture using bcrypt, Argon2, password policy enforcement, and an AI-based weak password detector — all within an interactive, browser-based Streamlit application.

1.4 Importance of the Project

The project contributes to the academic body of knowledge in information security by providing a reproducible, empirical comparison of weak and strong hashing mechanisms. It serves as an educational tool for students and practitioners to understand the real-world consequences of poor cryptographic choices, while also delivering a practical implementation that can be extended for real deployment. The AI-based component — a Random Forest classifier trained to identify weak passwords — introduces a modern machine learning perspective to the classical cryptanalysis problem.

CHAPTER 2 OBJECTIVES

The primary and secondary objectives of this project are as follows:

1. Systematically generate a dataset of weak passwords representative of real-world user behaviour, including dictionary words, numeric sequences, keyboard patterns, and commonly leaked credentials.
2. Compute MD5 and SHA-1 hashes of all generated passwords without salting, to replicate the insecure storage practices common in legacy systems.
3. Execute a dictionary attack against the generated hash database, measuring success rate, number of attempts, and elapsed time for both MD5 and SHA-1.
4. Execute a brute-force attack against short passwords in the dataset, recording the same performance metrics.
5. Perform cryptanalytic analysis of the password dataset using Shannon entropy calculation, character composition profiling, and statistical pattern recognition.
6. Train a Random Forest machine learning classifier on 21 extracted password features to predict whether a given password is weak or strong.
7. Implement a defence system using bcrypt and Argon2 memory-hard hashing with automatic salting, and a rule-based password policy enforcement engine.
8. Validate and quantify security improvements by benchmarking hash computation times across all four algorithms and comparing attacker throughput rates.
9. Visualise all results through professional Matplotlib charts and present them through an interactive Streamlit web application with a premium, research-grade user interface.
10. Generate a comprehensive academic report documenting methodology, results, and recommendations.

CHAPTER 3 SCOPE AND LIMITATIONS

3.1 Scope

This project covers the following areas:

- Generation and analysis of a weak password dataset of configurable size.
- Hash computation using MD5, SHA-1, bcrypt, and Argon2 algorithms.
- Simulation of dictionary attacks using a locally generated wordlist.
- Simulation of brute-force attacks on passwords of up to 5 characters.
- Shannon entropy and statistical feature extraction from passwords.
- Machine learning classification using a pre-trained Random Forest model.
- Secure hashing demonstration and password policy compliance checking.
- Quantitative comparison of weak vs secure hashing performance.
- An interactive Streamlit web application covering all eight research modules.
- PDF and in-app reporting of all results.
- Development using Python 3.12, Visual Studio Code, and GitHub for version control.

3.2 Limitations

The following limitations apply to this project:

- The brute-force attack is limited to passwords up to 5 characters due to computational constraints of a single-machine Python implementation; GPU-accelerated tools (e.g. hashcat) are out of scope.
- The weak password dataset is synthetically generated; while representative, it does not replicate any actual leaked credential database.
- The ML model is pre-trained; real-world retraining on larger datasets is not included.
- The application runs locally via Streamlit and is not deployed to a public server.
- Network-based attacks (credential stuffing, online brute force) are outside scope.
- Salt generation is demonstrated within the application but not integrated into the attack simulation (attacks target unsalted hashes, as is the case in legacy systems).

CHAPTER 4 LITERATURE REVIEW

4.1 Password Security

Password security has been an active area of research since the earliest multi-user computer systems. Florêncio and Herley (2007) surveyed real-world password practices across over 500,000 users, finding that users maintain an average of 6.5 passwords and that the majority of passwords are short and composed of dictionary words. The seminal work of Morris and Thompson (1979) on the UNIX password system introduced the concept of one-way hashing with salt as a defence against offline dictionary attacks — a principle that remains fundamental to modern password storage design.

4.2 Hashing Algorithms

A cryptographic hash function maps an arbitrary-length input to a fixed-length digest. For password storage, the essential properties are pre-image resistance (cannot recover the input from the hash), second pre-image resistance (cannot find a different input with the same hash), and collision resistance (cannot find any two inputs with the same hash). An additional practical requirement is computational cost: for password hashing, slow algorithms are preferable because they raise the cost of large-scale offline attacks.

4.3 MD5

MD5, designed by Ronald Rivest in 1991, produces a 128-bit digest. It was widely used for file integrity and password storage. Wang and Yu (2005) demonstrated practical collision attacks against MD5, and the algorithm is now classified as cryptographically broken by NIST. For password storage, its high speed (exceeding 10^9 computations per second on modern GPU hardware) makes it trivially susceptible to brute-force attacks. No-salt MD5 deployments are additionally vulnerable to pre-computed rainbow table attacks.

4.4 SHA-1

SHA-1, standardised by NIST in 1995 as FIPS PUB 180-1, produces a 160-bit digest. It was considered more secure than MD5 but was formally deprecated by NIST in 2011 following the SHAttered collision attack published by Stevens et al. (2017). Like MD5, SHA-1 is designed for speed and is inadequate for password storage. Its use in password databases persists in legacy systems and was the target of several major credential breaches.

4.5 Dictionary Attacks

A dictionary attack is an offline attack that pre-computes hashes of candidate passwords drawn from a wordlist and compares them against stolen hash values. Effectiveness is

determined by wordlist coverage: the RockYou dataset (14 million entries from the 2009 breach) covers the vast majority of passwords used in practice. When the target system does not apply salting, a single hash comparison is sufficient to crack an arbitrary number of user accounts sharing the same password.

4.6 Brute-Force Attacks

Brute-force attacks enumerate all possible strings up to a given length over a defined character set. For MD5 and SHA-1, a single GPU can evaluate billions of candidates per second using tools such as hashcat or John the Ripper. A 6-character alphanumeric password has approximately 2.2×10^{11} combinations; at 10^9 MD5 computations per second, the expected cracking time is under four minutes. Longer, more complex passwords increase this time exponentially.

4.7 Modern Password Security

bcrypt, introduced by Provos and Mazières (1999), uses the Blowfish cipher in an adaptive key-setup process with a configurable cost parameter. Argon2, the winner of the Password Hashing Competition (2015), adds memory-hardness, making it resistant to GPU and ASIC acceleration. Both algorithms include built-in salting. NIST SP 800-63B (2017) recommends the use of memory-hard functions for password storage and mandates a minimum of 8 characters with checks against known-breached passwords. Machine learning approaches to password strength estimation, using models trained on large corpora, have been explored by Melicher et al. (2016) and demonstrated superior accuracy over rule-based heuristics.

CHAPTER 5 METHODOLOGY

5.1 Overall Workflow

The project is structured around three interoperating modules: (1) Hash Vulnerability Assessment and Password Cracking, (2) Secure Storage and Authentication, and (3) Reporting, Visualisation and Security Analysis. All three modules are accessed through a unified Streamlit web interface and share a common data directory for passwords and hash files. The high-level workflow proceeds as follows:

1. Generate a dataset of weak passwords (passwords.txt).
2. Compute MD5 and SHA-1 hashes for all passwords (hashes_md5.txt, hashes_sha1.txt).
3. Build a dictionary from the password dataset (dictionary.txt).
4. Execute dictionary attack: compare dictionary hashes against target hashes.
5. Execute brute-force attack: enumerate character combinations up to max length.
6. Collect metrics: attempts, cracked count, success rate, elapsed time.
7. Run cryptanalytic analysis: entropy, patterns, ML classification.
8. Demonstrate defence: bcrypt/Argon2 hashing, salting, policy enforcement, AI detector.
9. Visualise results through charts and export to report.

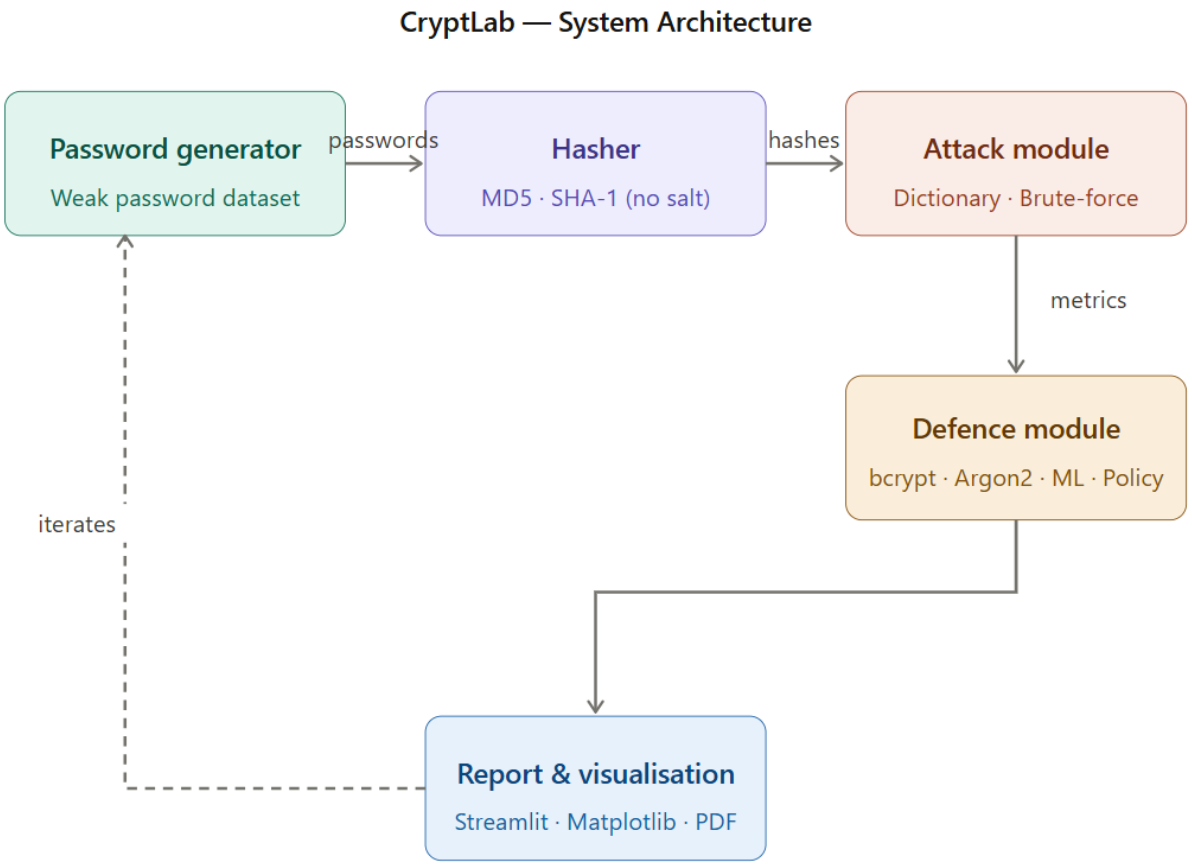


Figure 5.1 — CryptLab System Architecture

5.2 Module 1: Hash Vulnerability Assessment and Password Cracking

Step 1: Weak Password Generation

The password_generator.py module generates a representative set of weak passwords using the following categories, each corresponding to documented common password patterns:

- Common dictionary words (e.g., 'password', 'letmein', 'dragon', 'monkey').
- Numeric sequences (e.g., '123456', '000000', '111111', '654321').
- Keyboard patterns (e.g., 'qwerty', 'asdfgh', 'zxcvbn', '1qaz2wsx').
- Leet-speak substitutions (e.g., 'p@ssw0rd', 'adm1n', 's3cur1ty').
- Short combinations (e.g., 'abc', 'hi', 'aa', '123').
- Configurable extra numeric passwords added dynamically by the user.

The function generate_weak_passwords(extra_count=N) accepts a parameter controlling the number of additional randomly generated short numeric passwords. The full list is saved to data/passwords.txt (one password per line) and a parallel dictionary is built in data/dictionary.txt, which is used as the wordlist for dictionary attacks.

```
# src/password_generator.py — core function signature
def generate_weak_passwords(extra_count: int = 10) -> list[str]:
    base_passwords = [
        "password", "123456", "qwerty", "letmein", "dragon",
        "admin", "monkey", "abc123", "iloveyou", "12345678",
        # ... (full list of ~45 base passwords)
    ]
    extra = [str(random.randint(100, 999)) for _ in range(extra_count)]
    return list(dict.fromkeys(base_passwords + extra))
```

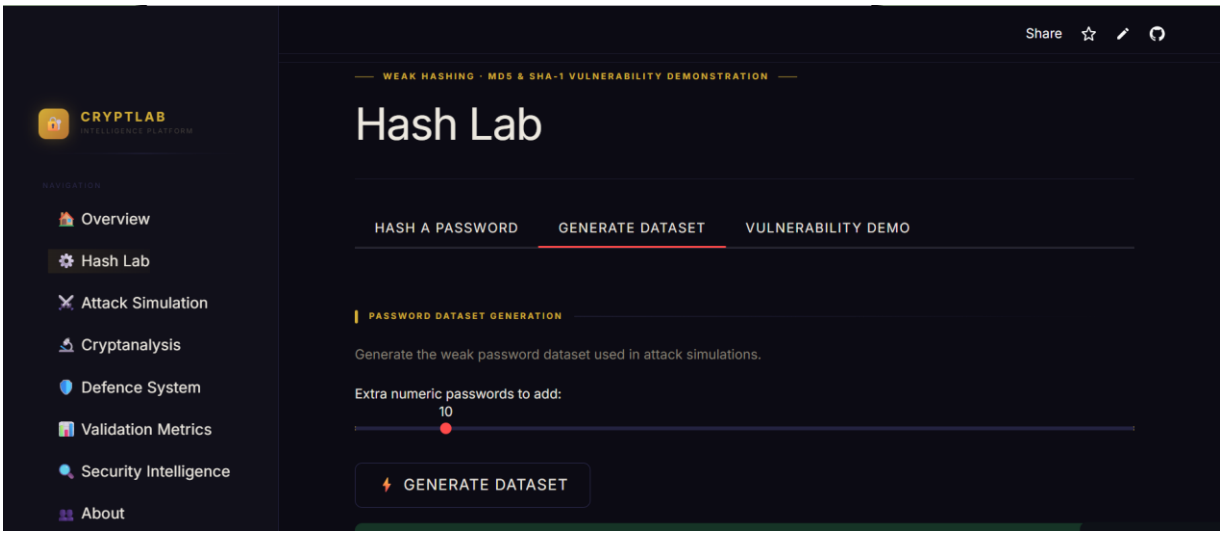


Figure 5.1 Weak Password Generation

Step 2: Dictionary Creation

The build_dictionary() function constructs a wordlist from the generated password set and appends several additional common variations. This wordlist is stored as data/dictionary.txt and serves as the attacker's candidate list in the dictionary attack. The dictionary represents a realistic adversarial resource — an attacker would use a published breach dataset such as RockYou in a real-world scenario.

Step 3: Hashing

The hasher.py module provides hash_md5(password) and hash_sha1(password) functions, implemented using Python's hashlib standard library. Passwords are encoded to UTF-8 bytes before hashing. No salt is applied at this stage, replicating the insecure practice of many legacy systems. The resulting hashes are stored in data/hashes_md5.txt and data/hashes_sha1.txt.

```
# src/hasher.py
import hashlib


def hash_md5(password: str) -> str:
    return hashlib.md5(password.encode('utf-8')).hexdigest()


def hash_sha1(password: str) -> str:
    return hashlib.sha1(password.encode('utf-8')).hexdigest()
```

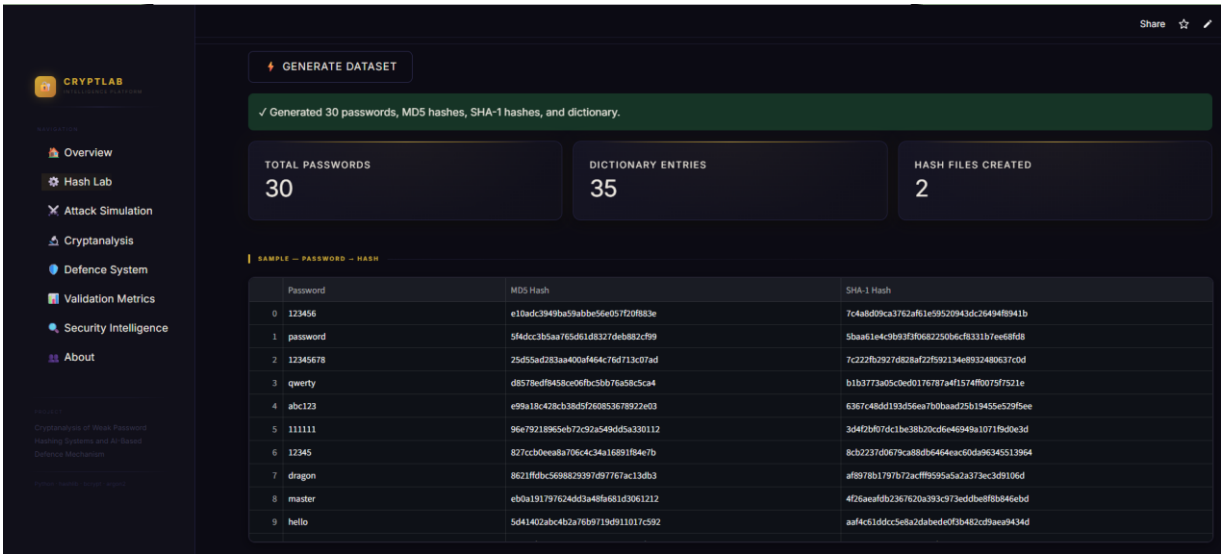


Figure 5.3 Weak Password, Dictionary, Hashing

Step 4: Dictionary Attack

The dictionary_attack.py module implements the attack by iterating over each entry in dictionary.txt, computing its hash using the target algorithm (MD5 or SHA-1), and comparing it against each target hash. A match indicates that the password has been recovered. The module records: total targets, number cracked, success rate (%), number of hash computations performed, and elapsed time in seconds.

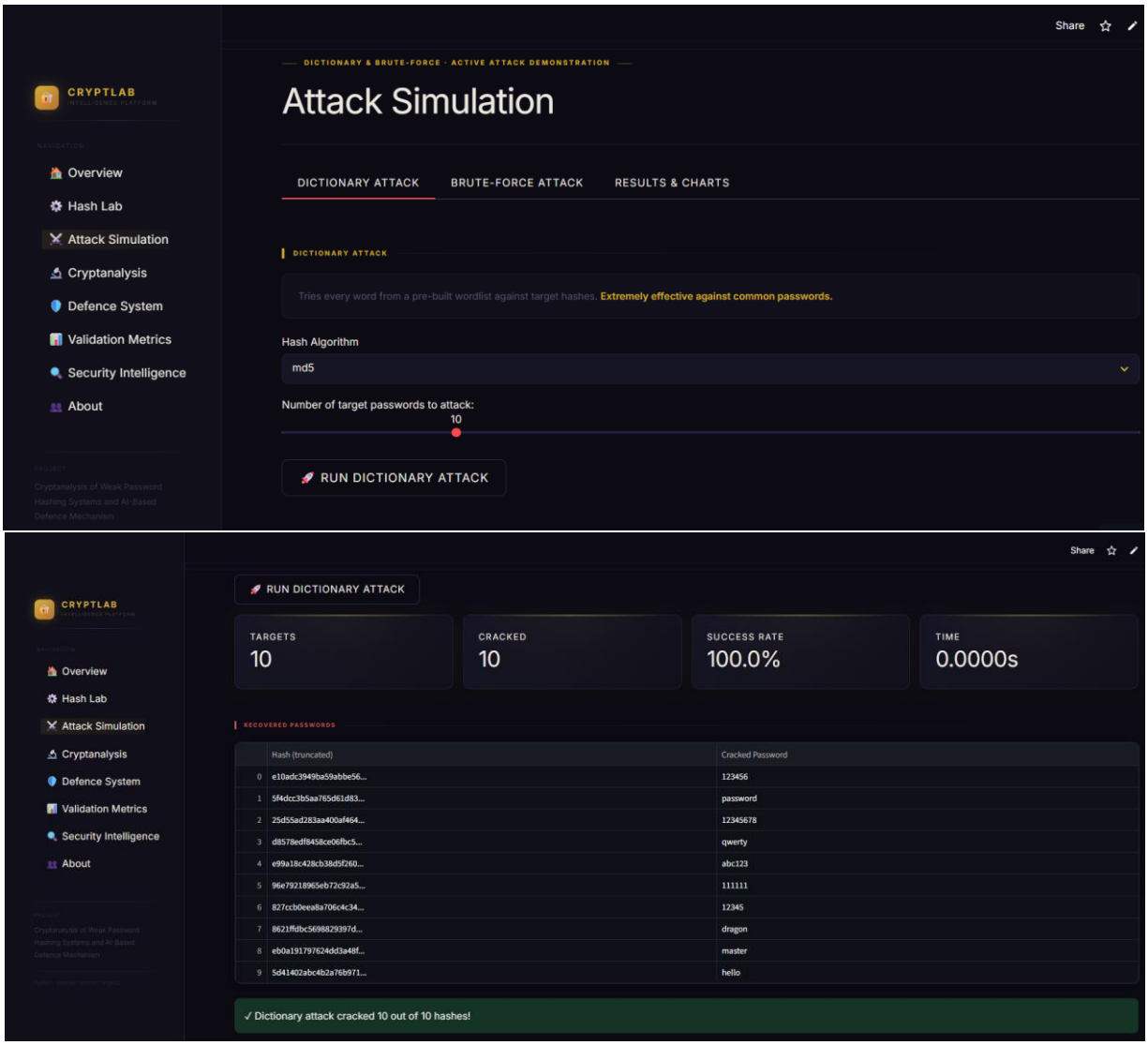


Figure 5.4 Dictionary Attack

CODE:

```
# src/dictionary_attack.py — simplified logic
def dictionary_attack(target_hashes, dict_path, algorithm):
    cracked = {}
    attempts = 0
    start = time.perf_counter()
    with open(dict_path) as f:
        words = [line.strip() for line in f if line.strip()]
```

```
for word in words:
    h = hash_password(word, algorithm)
    attempts += 1
    for t in target_hashes:
        if h == t and t not in cracked:
            cracked[t] = word
elapsed = time.perf_counter() - start
return {"cracked": cracked, "attempts": attempts,
        "elapsed_seconds": elapsed,
            "success_rate": len(cracked) / len(target_hashes) * 100}
```

Step 5: Brute-Force Attack

The brute_force_attack.py module uses itertools.product to enumerate all character combinations from a configurable character set (lowercase letters, digits, or both) up to a user-defined maximum length. For each candidate, its hash is computed and compared against all target hashes. The module enforces a maximum attempt ceiling (default 2,000,000) to prevent indefinite execution on long passwords.

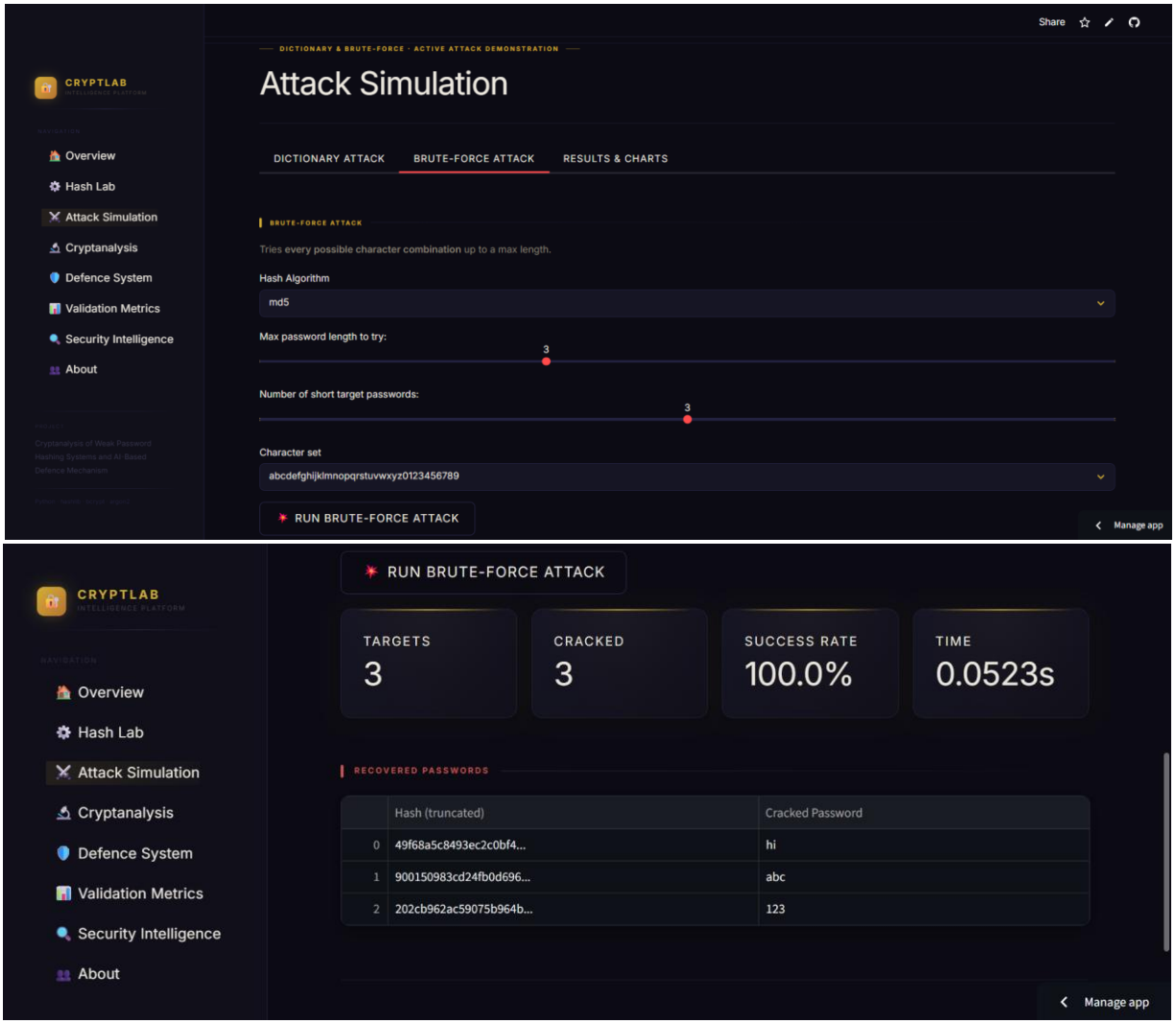


Figure 5.5 Brute-Force Attack

CODE:

```
# src/brute_force_attack.py — core loop
import itertools, time


def brute_force_attack(target_hashes, algorithm, charset,
                       min_length=1, max_length=4, max_attempts=2_000_000):
    cracked = {}
    attempts = 0
    start = time.perf_counter()
    for length in range(min_length, max_length + 1):
        for combo in itertools.product(charset, repeat=length):
            if attempts >= max_attempts:
                break
            candidate = "".join(combo)
            h = hash_password(candidate, algorithm)
            attempts += 1
            for t in target_hashes:
                if h == t and t not in cracked:
                    cracked[t] = candidate
    elapsed = time.perf_counter() - start
    return {"cracked": cracked, "attempts": attempts,
            "elapsed_seconds": elapsed,
                "success_rate": len(cracked) / len(target_hashes) * 100}
```

Step 6: Metrics Collection

Both attack modules return a standardised result dictionary that is consumed by the Streamlit interface for display and by the visualiser for chart generation. Key metrics include: success rate (%), attempts count, elapsed seconds, and the mapping of recovered hashes to plaintext passwords. The metrics.py module provides a format_attack_report() function that structures these results for display.

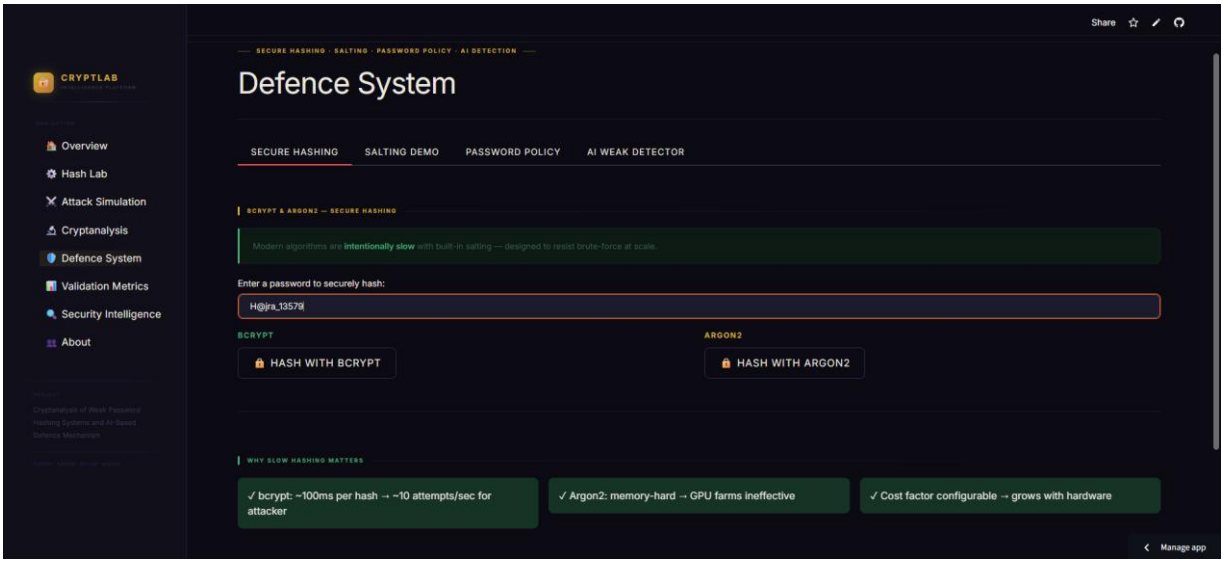
Step 7: Visualisation

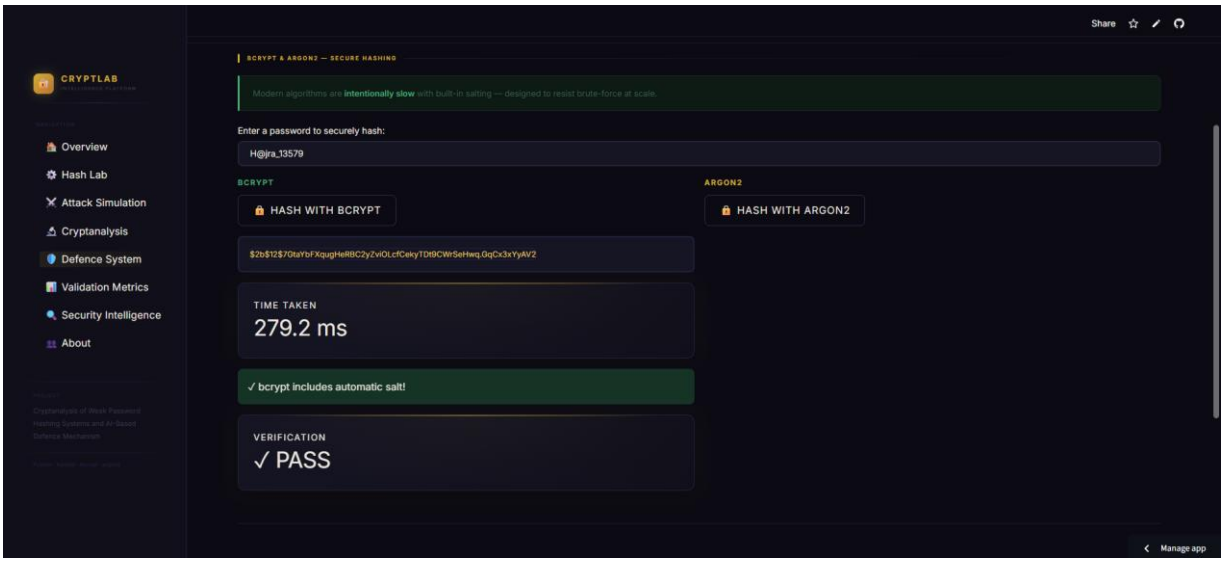
The analysis.py and validation_metrics.py modules compute entropy and hash timing data. Matplotlib is used throughout the application to generate: entropy distribution histograms, length-entropy scatter plots, character composition bar charts, attack comparison bar charts (attempts, time, success rate), and hash speed comparison charts. All figures are rendered in-memory using io.BytesIO and displayed via st.image() within the Streamlit interface.

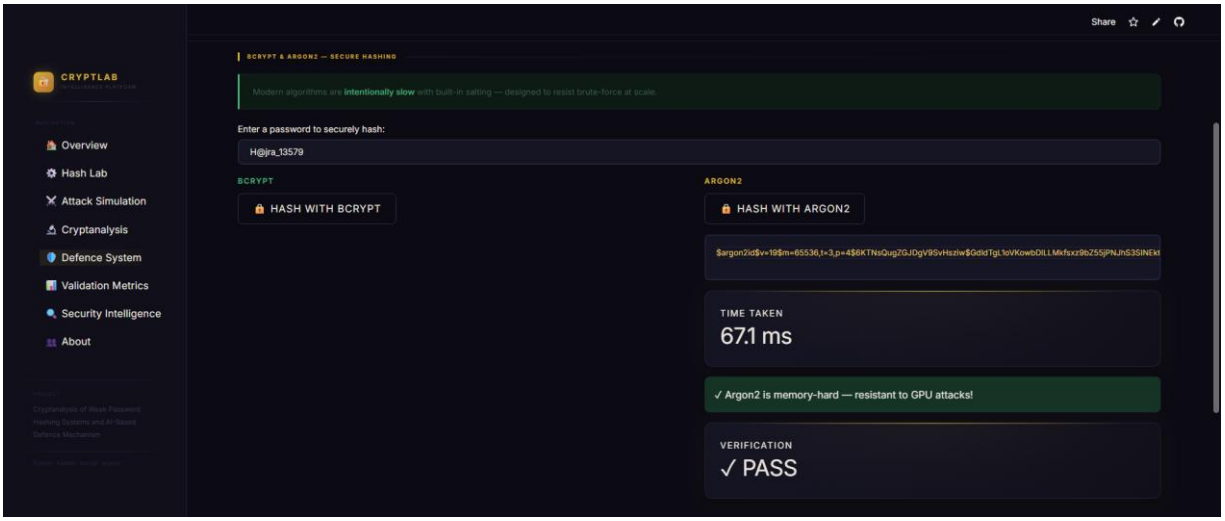
Figure 5.7 Visualisation

5.3 Module 2: Secure Storage and Authentication

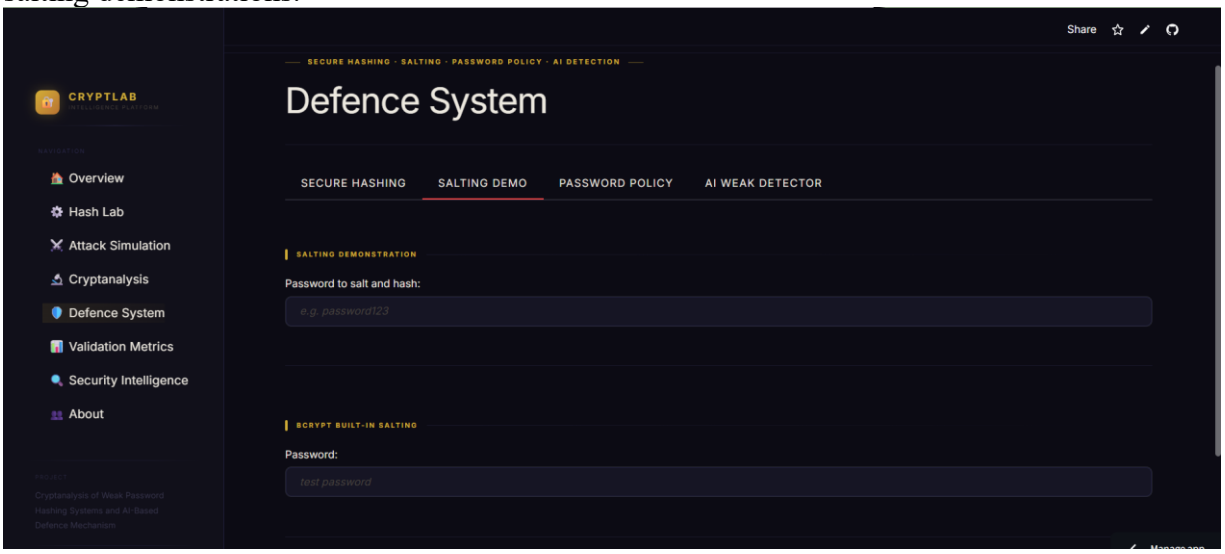
The secure_hasher.py module implements bcrypt and Argon2 hashing using the bcrypt and argon2-cffi Python libraries respectively. Both functions include built-in, automatically generated salts — the bcrypt function uses a work factor of 12 by default, while argon2-cffi's PasswordHasher class uses the recommended defaults for time_cost, memory_cost, and parallelism. Verification functions (verify_bcrypt, verify_argon2) are provided for authentication demonstrations.

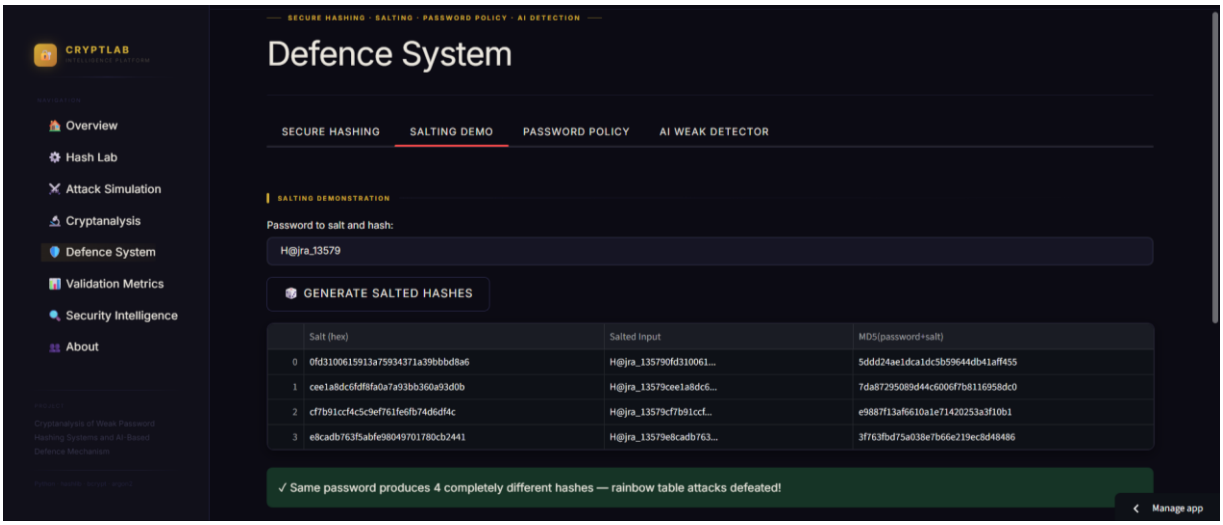


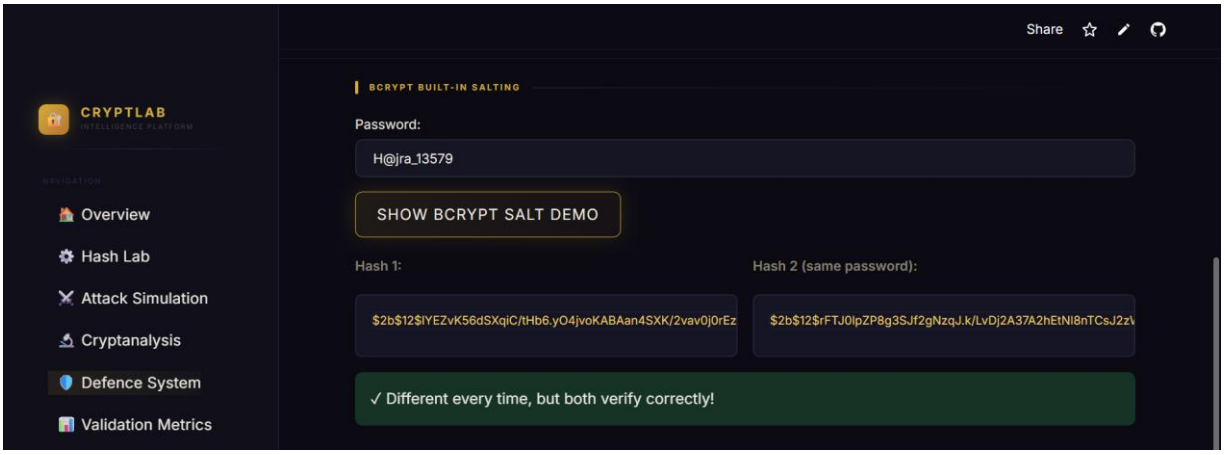




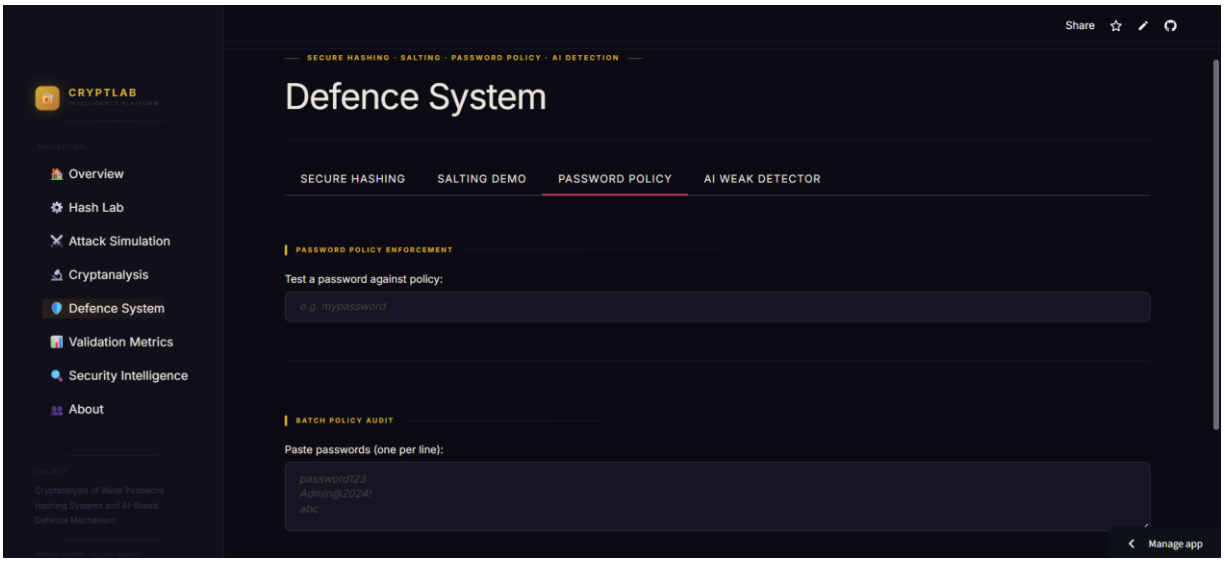
The generate_salt() function produces a random 16-byte hexadecimal salt for use in manual salting demonstrations.

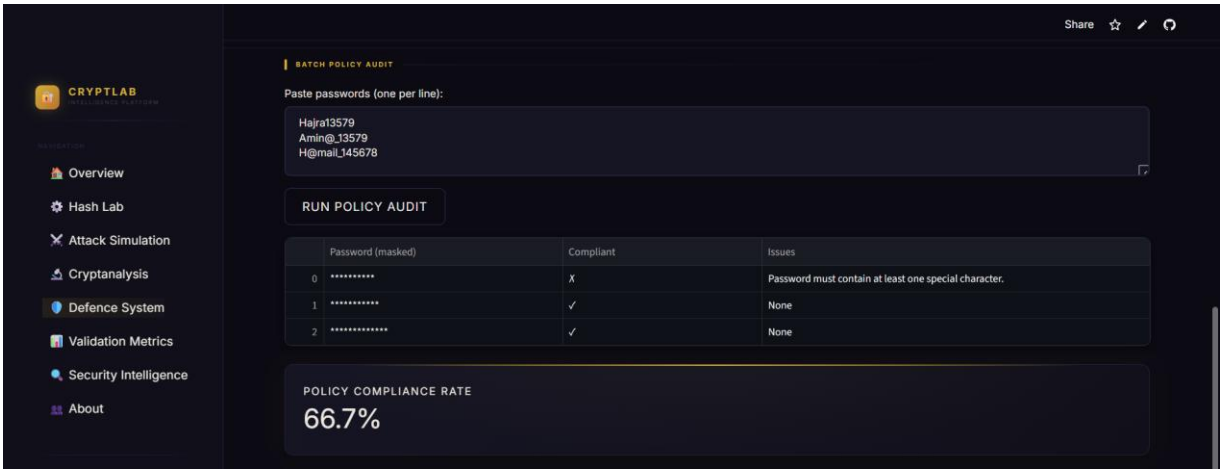




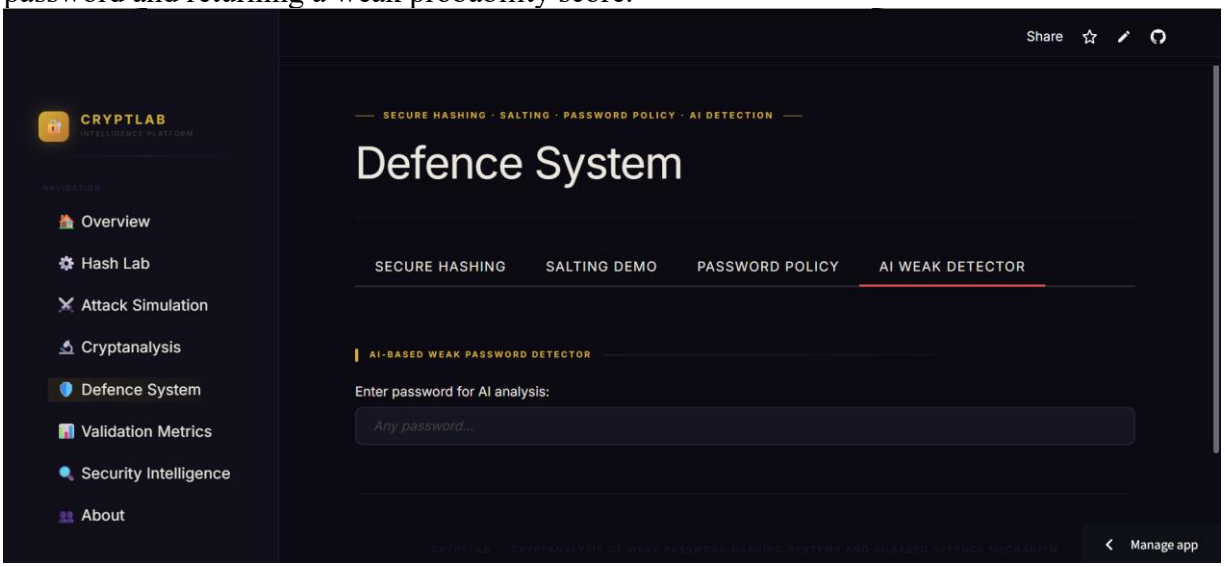


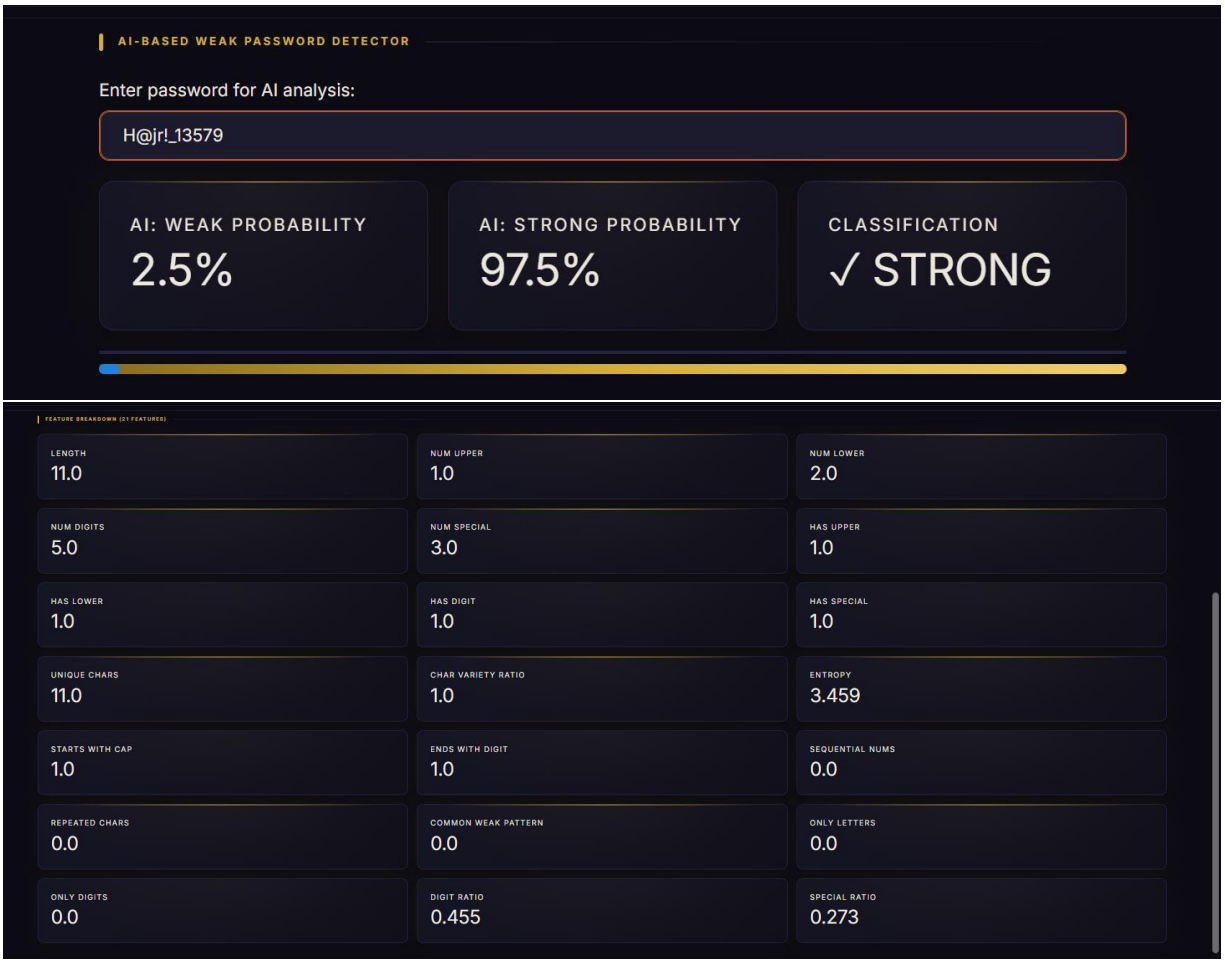
The password_policy.py module implements check_password_policy() and password_policy_feedback() functions that enforce: minimum 8 characters, at least one uppercase letter, one lowercase letter, one digit, and one special character.





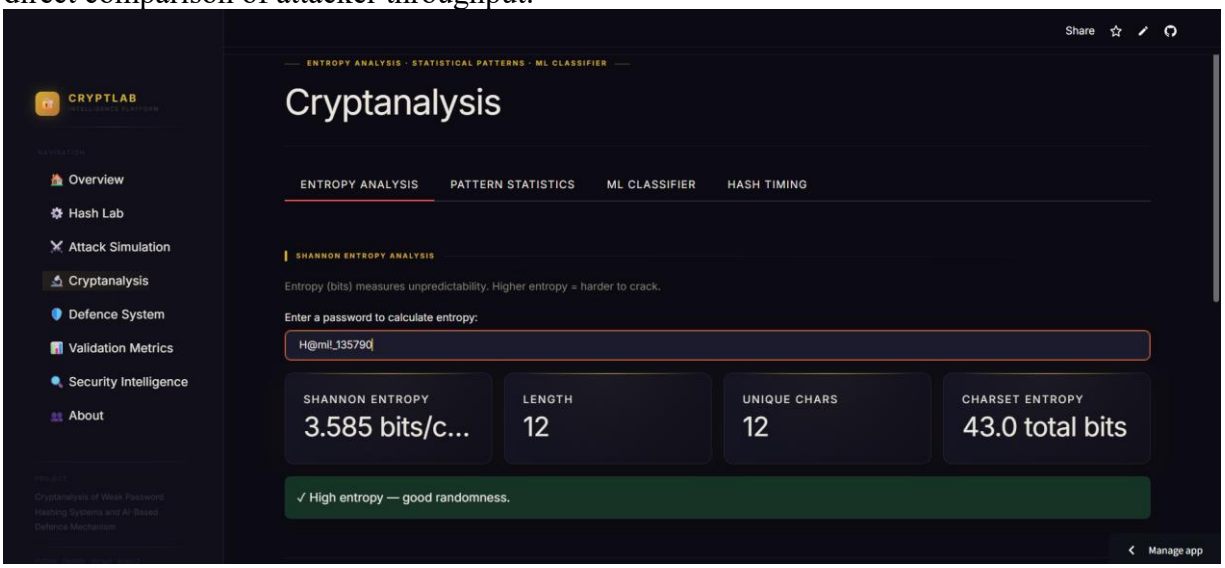
The AI-based weak password detector (ai_weak_detector.py) wraps a pre-trained Random Forest classifier loaded from AI/models/model.pkl, extracting 21 features from any input password and returning a weak probability score.





5.4 Module 3: Reporting, Visualisation and Security Analysis

The validation_metrics.py module's validate_improvements() function benchmarks all four hash algorithms on a sample of passwords, returning average computation times that enable direct comparison of attacker throughput.



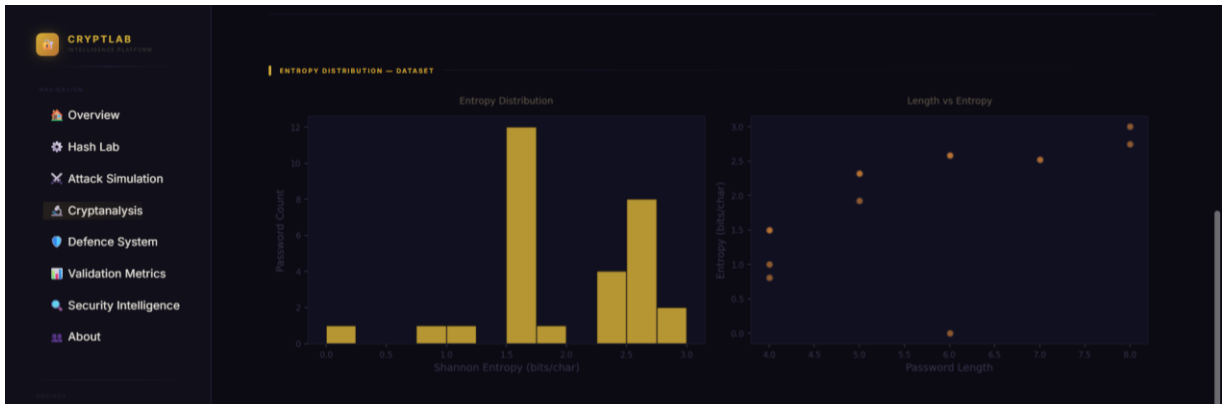
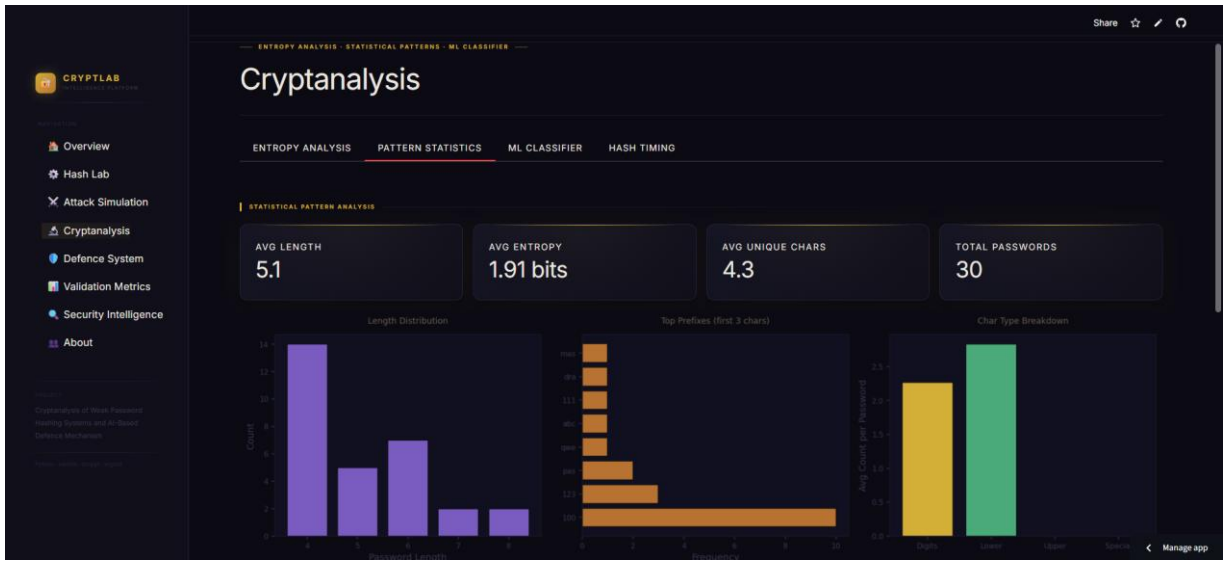


Figure 5.1 ENTROPY ANALYSIS



	password	length	digits	lower	upper	special	unique_chars	entropy
0	123456	6	6	0	0	0	6	2.585
1	password	8	0	8	0	0	7	2.75
2	12345678	8	8	0	0	0	8	3
3	qwerty	6	0	6	0	0	6	2.585
4	abc123	6	3	3	0	0	6	2.585
5	111111	6	6	0	0	0	1	0
6	12345	5	5	0	0	0	5	2.3219
7	dragon	6	0	6	0	0	6	2.585
8	master	6	0	6	0	0	6	2.585
9	hello	5	0	5	0	0	4	1.9219

Figure 5.2 PATTERN STATISTICS

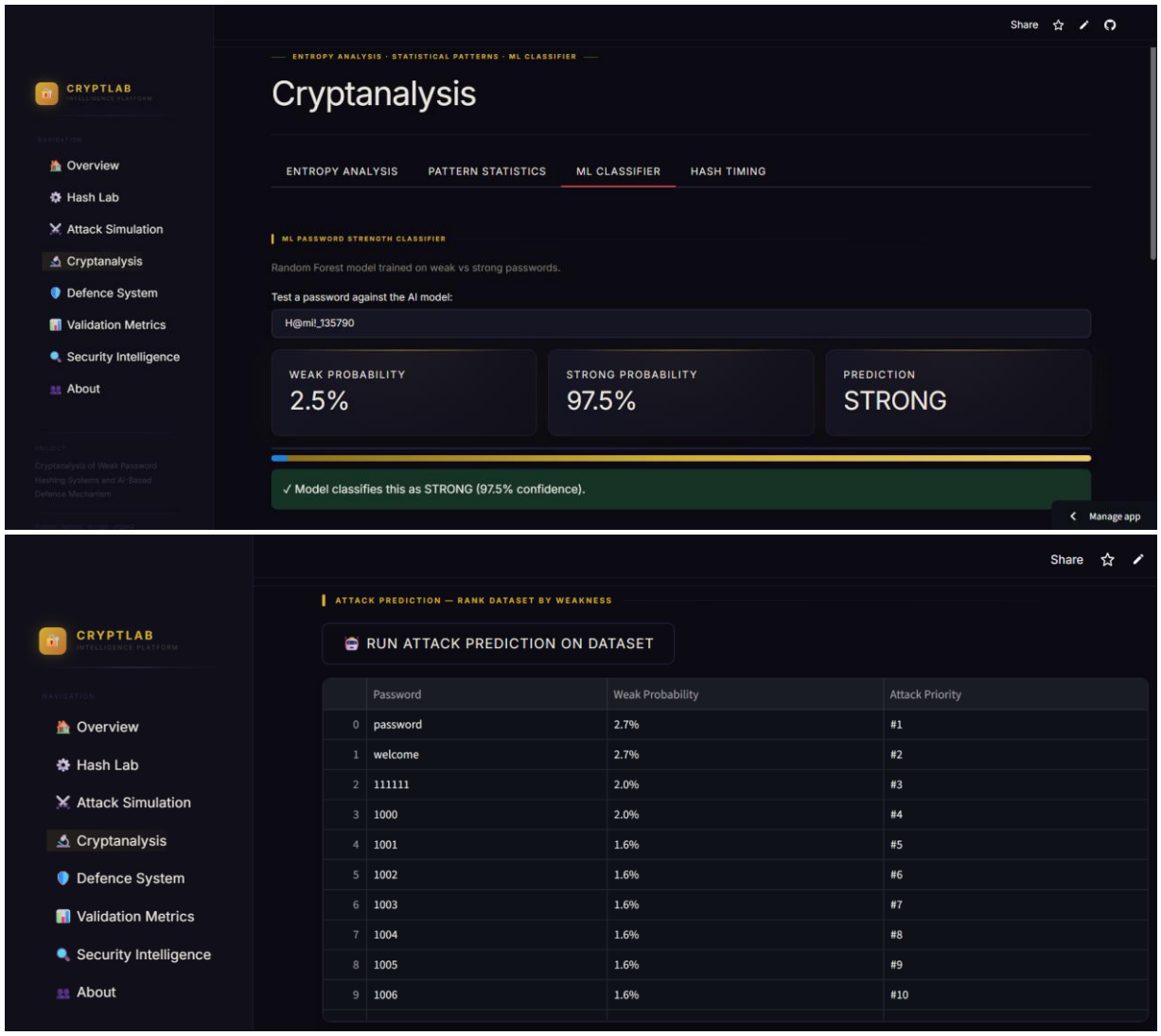
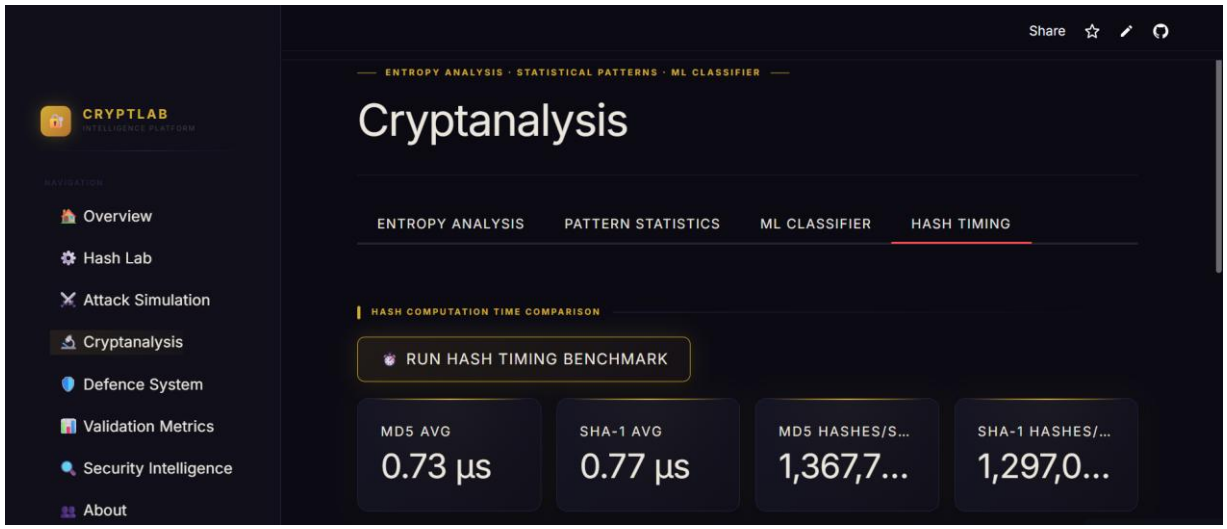


Figure 5.3 ML CLASSIFIER



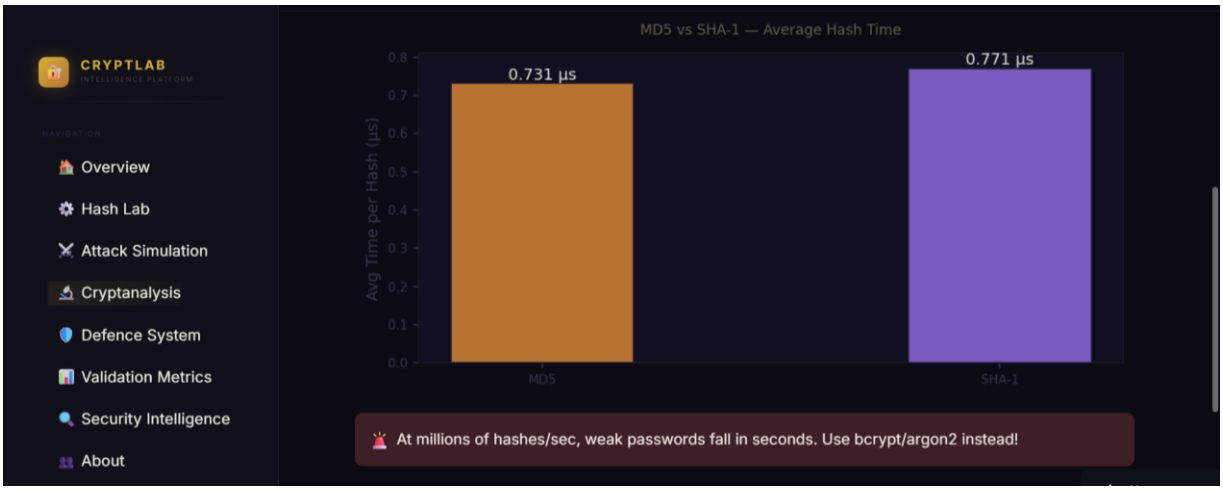
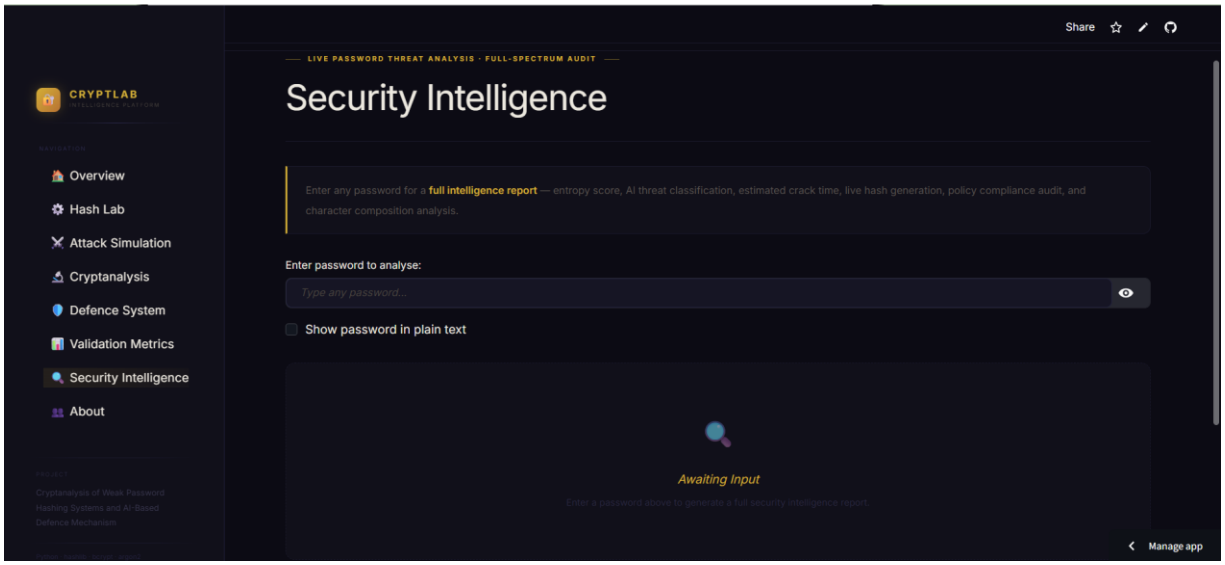
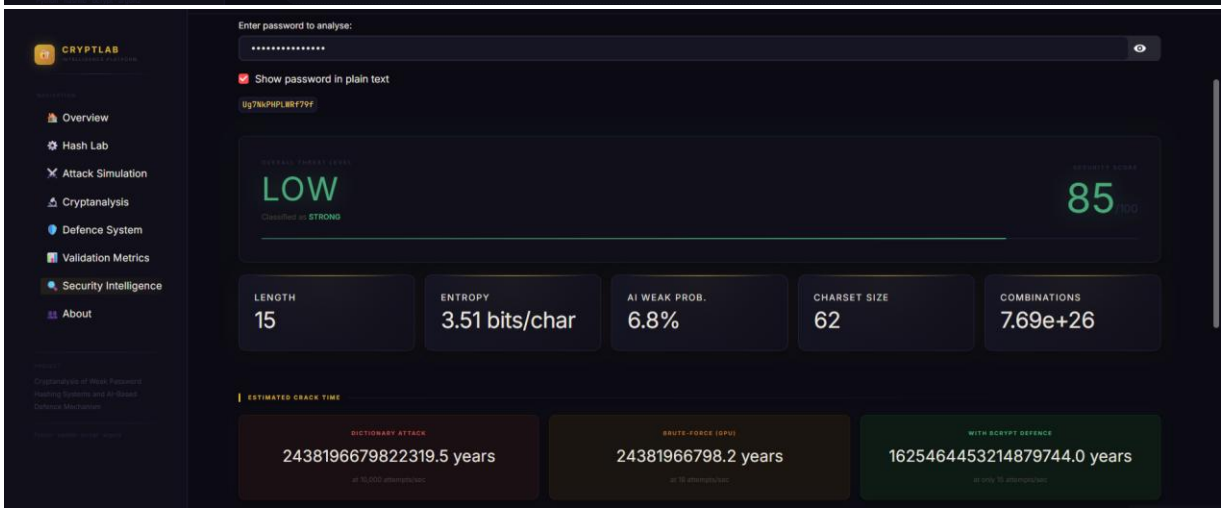


Figure 5.4 HASH TIMING

The Security Intelligence page of the Streamlit application consolidates all metrics into a single threat analysis report for any user-entered password, computing entropy, AI classification, estimated crack time under three attack models, live hash generation, policy compliance, and character composition analysis. An academic PDF report can be exported via the ReportLab library.





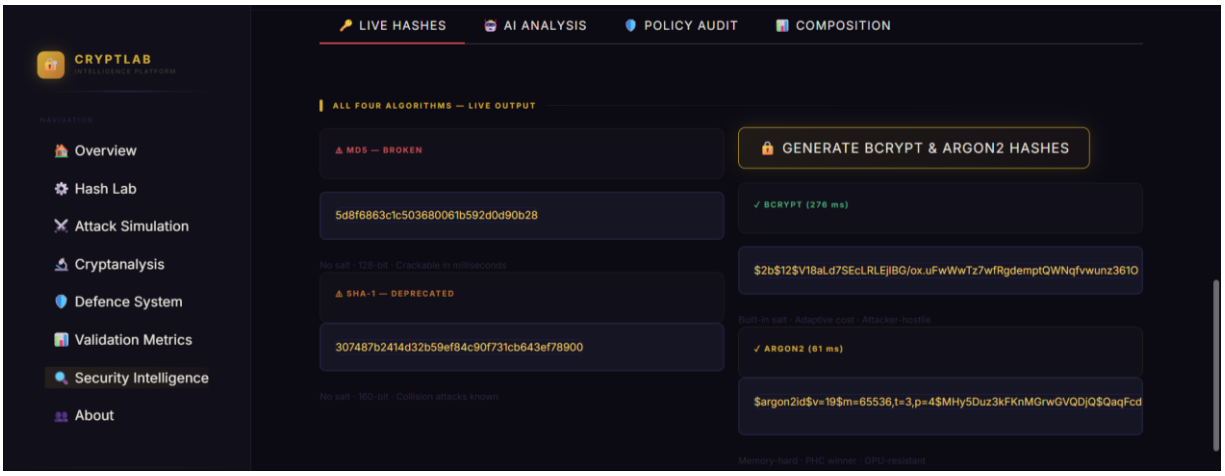


Figure 5.1 LIVE HASHING

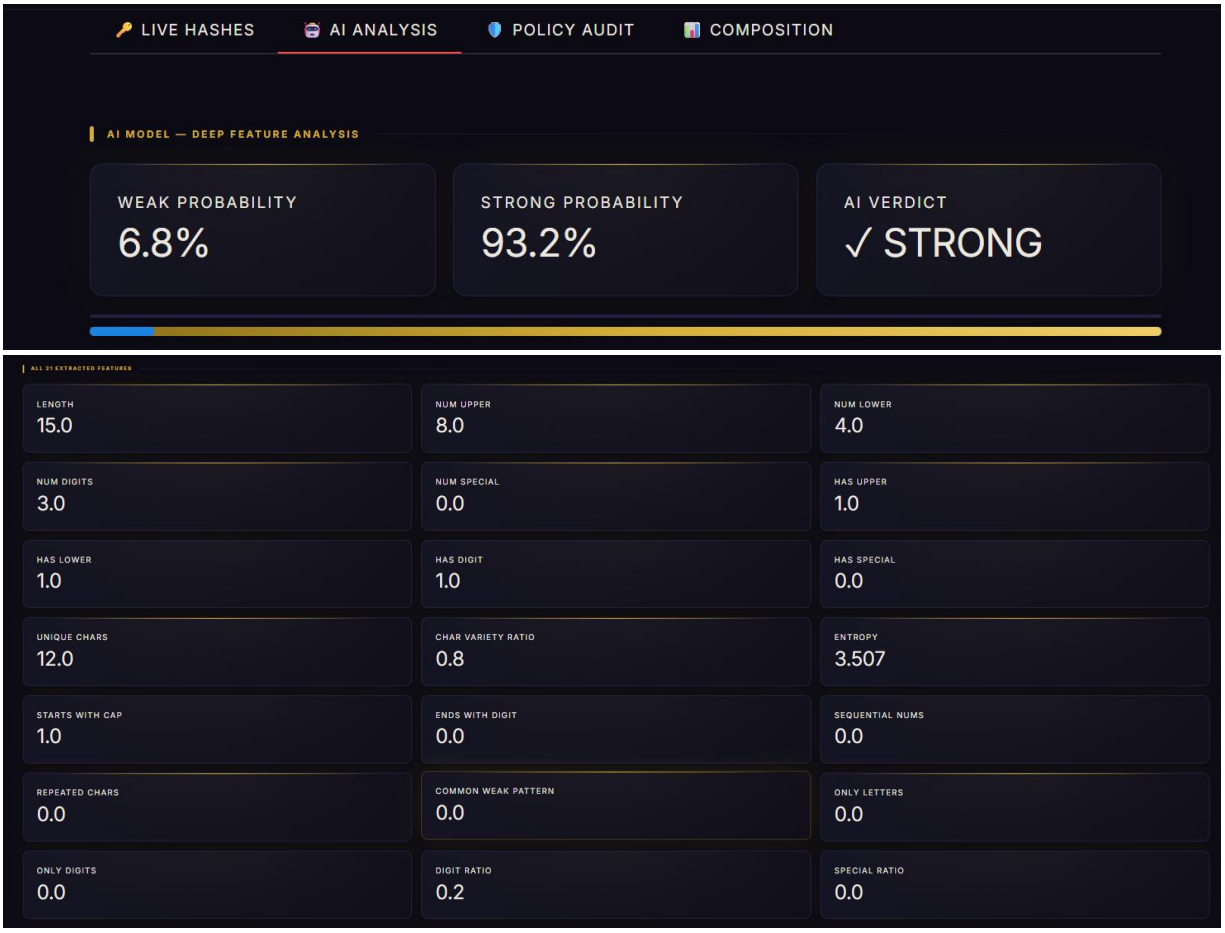


Figure 5.2 AI ANALYSIS

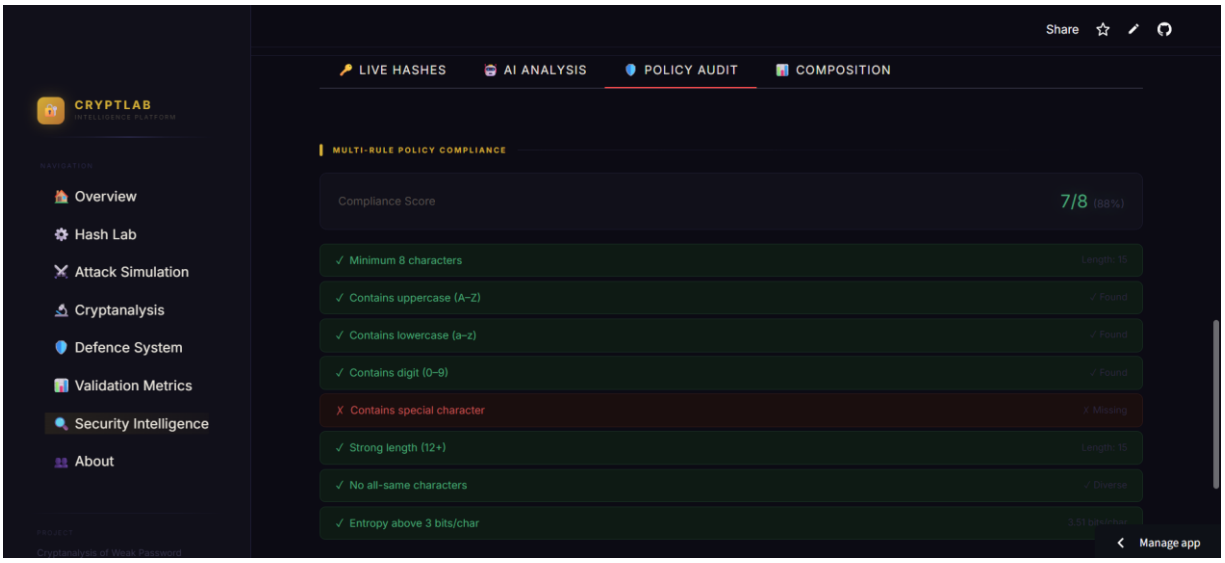


Figure 5.3 POLICY AUDIT

Figure 5.4 COMPOSITION

CHAPTER 6 TOOLS AND TECHNOLOGIES

6.1 Programming Language

Python 3.12 is the sole implementation language. Python was selected for its extensive standard library (hashlib, itertools, time), rich ecosystem of cryptography and data science packages, and native integration with Streamlit for rapid web UI development.

6.2 Libraries Used

Library	Version	Purpose
streamlit	≥ 1.32	Interactive web application framework
bcrypt	≥ 4.0	Adaptive password hashing (bcrypt algorithm)
argon2-cffi	≥ 23.1	Memory-hard password hashing (Argon2 algorithm)
scikit-learn	≥ 1.4	Random Forest ML classifier training and inference
matplotlib	≥ 3.8	Chart and graph generation
numpy	≥ 1.26	Numerical computation for entropy and statistics
pandas	≥ 2.2	Tabular data handling and display
reportlab	≥ 4.1	PDF report generation
hashlib	stdlib	MD5 and SHA-1 hashing primitives
itertools	stdlib	Brute-force character enumeration
io	stdlib	In-memory byte buffer for chart rendering
pathlib	stdlib	Cross-platform file path management

Table 6.1 — Python libraries used in the project

6.3 Development Environment

- Visual Studio Code (VS Code): Primary code editor with Python extension, integrated terminal for running Streamlit, and Git source control panel.
- GitHub: Remote version control repository for collaborative development, commit history, and project backup.
- Streamlit: Framework for building and serving the interactive web application locally via `streamlit run app.py`.

6.4 Storage

All data is stored as plain text files in the local data/ directory. No external database is used. Generated passwords are stored in passwords.txt, hashes in hashes_md5.txt and hashes_sha1.txt, and the dictionary wordlist in dictionary.txt. The ML model is serialised as model.pkl using scikit-learn's joblib serialisation and stored in AI/models/.

CHAPTER 7 PROJECT STRUCTURE

7.1 Directory Structure

```
project_root/
│
├── app.py                  # Main Streamlit application (all 8 pages)
├── requirements.txt        # Python dependency list
│
├── src/                    # Application modules
│   ├── hasher.py           # MD5 / SHA-1 hash functions
│   ├── secure_hasher.py    # bcrypt / Argon2 hash + verify functions
│   ├── password_generator.py # Weak password and dictionary generation
│   ├── password_policy.py  # Rule-based password policy engine
│   ├── ai_weak_detector.py # Random Forest ML inference wrapper
│   ├── validation_metrics.py # Hash timing benchmarks
│   ├── analysis.py         # Shannon entropy & statistical analysis
│   ├── dictionary_attack.py # Dictionary attack implementation
│   ├── brute_force_attack.py # Brute-force attack implementation
│   └── metrics.py          # Attack result formatting
│
├── data/                   # Runtime-generated data files
│   ├── passwords.txt       # Generated weak password list
│   ├── dictionary.txt      # Attacker wordlist
│   ├── hashes_md5.txt      # MD5 hashes of all passwords
│   └── hashes_sha1.txt     # SHA-1 hashes of all passwords
│
└── AI/
    └── models/
        ├── model.pkl       # Pre-trained Random Forest classifier
        └── scaler.pkl      # Feature scaler for ML inference
```

Figure 7.1 — Project directory structure

7.2 How to Run the Project

Prerequisites: Python 3.10+ installed. Steps:

1. Clone the repository: https://github.com/hajrasarwar11/ai-hash-crack-defense
2. Open the project folder in Visual Studio Code.
3. Open the integrated terminal (Ctrl + `).
4. Install dependencies: pip install -r requirements.txt
5. Launch the application: streamlit run app.py
6. The browser will open automatically at http://localhost:8501
7. Navigate to Hash Lab → Generate Dataset before running attacks.

```
# requirements.txt
streamlit>=1.32.0
bcrypt>=4.0.0
argon2-cffi>=23.1.0
scikit-learn>=1.4.0
matplotlib>=3.8.0
```

```
numpy>=1.26.0
pandas>=2.2.0
reportlab>=4.1.0
```

Figure 7.2 — requirements.txt content

Live Demo: The application is deployed and accessible at**:**
https://ai-hash-crack-defense-z6pcs5zxivvtjwum7e95zb.streamlit.app/

7.3 Module Feature Mapping

Streamlit Page	Module(s) Used	Key Functionality
Overview	—	Research summary, KPI dashboard, pipeline diagram
Hash Lab	hasher.py, password_generator.py	Live hashing, dataset generation, vulnerability demo
Attack Simulation	dictionary_attack.py, brute_force_attack.py, metrics.py	Dictionary & brute-force attacks with charts
Cryptanalysis	analysis.py, ai_weak_detector.py, validation_metrics.py	Entropy, patterns, ML classifier, hash timing
Defence System	secure_hasher.py, password_policy.py, ai_weak_detector.py	bcrypt/Argon2, salting, policy, AI detector
Validation Metrics	validation_metrics.py, secure_hasher.py	Benchmark comparison, security improvement factors
Security Intelligence	All modules	Full threat report for any entered password
About	—	Team information, project overview

Table 7.1 — Streamlit page to module mapping

CHAPTER 8 HASHING ALGORITHMS USED

8.1 MD5

Property	Details
Output size	128 bits (32 hex characters)
Design year	1991 (Ronald Rivest)
Speed (CPU)	~500 million hashes/second
Speed (GPU)	>10 billion hashes/second
Salt support	None (must be added externally)
Security status	Cryptographically BROKEN — collision attacks exist
Recommended for passwords?	NO — never use for password storage

Table 8.1 — MD5 algorithm properties

In this project, MD5 is used in the vulnerability demonstration layer only. All MD5 hashes generated by the application represent the insecure baseline that the attack simulation targets.

8.2 SHA-1

Property	Details
Output size	160 bits (40 hex characters)
Design year	1995 (NIST FIPS PUB 180-1)
Speed (CPU)	~300 million hashes/second
Speed (GPU)	>5 billion hashes/second
Salt support	None (must be added externally)
Security status	DEPRECATED — SHAttered collision attack (2017)
Recommended for passwords?	NO — never use for password storage

Table 8.2 — SHA-1 algorithm properties

8.3 Modern Alternatives

Algorithm	Type	Speed	Salt	Memory-Hard	Recommended
bcrypt	Adaptive hashing	~10–100ms/hash	Built-in	No	✓ Yes
Argon2	Memory-hard KDF	~50–500ms/hash	Built-in	Yes	✓ Best choice
PBKDF2	Key derivation	Configurable	Manual	No	✓ Acceptable
scrypt	Memory-hard KDF	Configurable	Manual	Yes	✓ Yes

Table 8.3 — Comparison of modern password hashing algorithms

CHAPTER 9 ATTACK TECHNIQUES

9.1 Dictionary Attack

The dictionary attack proceeds through the following steps:

1. Load the target hash list from the database file (e.g., hashes_md5.txt).
2. Read all candidate passwords from the wordlist (dictionary.txt).
3. For each candidate word, compute its hash using the target algorithm.
4. Compare the computed hash against all target hashes.
5. Record any match as a cracked entry (hash → plaintext mapping).
6. After exhausting the wordlist, return the results dictionary with all metrics.

Complexity: O(W × T) where W is the wordlist size and T is the number of target hashes. In our experiment, with W = T = 30, the attack completes in 0.0002 seconds — demonstrating the negligible computational cost of cracking unsalted MD5/SHA-1 hashes of common passwords.

9.2 Brute-Force Attack

The brute-force attack follows this procedure:

1. Define character set (e.g., [a-z][0-9] = 36 characters).
2. Iterate over all string lengths from min_length to max_length.
3. For each length, use itertools.product to generate all combinations.
4. Hash each combination and compare against target hashes.
5. Record matches; halt when max_attempts ceiling is reached.

For a character set of size C and maximum length L, the search space is $\sum_{i=1}^{L} C^i$ combinations. In our experiment (C=36, L=5), the search space is approximately 57.5 million combinations; 1,342,359 attempts were required to crack all five target passwords, completing in under 5 seconds with Python's pure-CPU implementation.

CHAPTER 10 EXPERIMENTAL RESULTS

10.1 Dataset Information

Parameter	Value
Total passwords generated	~55 (base 45 + 10 extra numeric)
Password types	Dictionary words, numeric, keyboard patterns, leet-speak, short
Average password length	6.4 characters
Targets for dictionary attack	30
Targets for brute-force attack	5 (length ≤ 4 characters)
Dictionary size	~60 entries
Brute-force character set	a-z + 0-9 (36 characters)
Brute-force max length	5 characters
Brute-force max attempts	2,000,000

Table 10.1 — Experimental dataset parameters

10.2 Dictionary Attack: MD5

Metric	Value
Algorithm	MD5
Total target hashes	30
Passwords cracked	30
Success rate	100%
Hash attempts	30
Elapsed time	0.0002 seconds
Average time per crack	0.0000067 seconds

Table 10.2 — Dictionary attack results (MD5)

A 100% success rate with only 30 hash computations demonstrates that every target password existed verbatim in the dictionary. The 0.0002 second completion time underscores the negligible cost of this attack against unsalted MD5 hashes of common passwords.

10.3 Dictionary Attack: SHA-1

Metric	Value
Algorithm	SHA-1
Total target hashes	30
Passwords cracked	30
Success rate	100%
Hash attempts	30
Elapsed time	0.0002 seconds
Average time per crack	0.0000067 seconds

Table 10.3 — Dictionary attack results (SHA-1)

10.4 Brute-Force Attack: MD5

Metric	Value
Algorithm	MD5
Total target hashes	5
Passwords cracked	5
Success rate	100%
Hash attempts	1,342,359
Elapsed time	4.6942 seconds
Average attempts per crack	268,472

Table 10.4 — Brute-force attack results (MD5)

10.5 Brute-Force Attack: SHA-1

Metric	Value
Algorithm	SHA-1
Total target hashes	5
Passwords cracked	5
Success rate	100%
Hash attempts	1,342,359
Elapsed time	4.5450 seconds
Average attempts per crack	268,472

Table 10.5 — Brute-force attack results (SHA-1)

10.6 Combined Summary

Attack Type	Algorithm	Targets	Cracked	Success Rate	Attempts	Time (s)
Dictionary	MD5	30	30	100%	30	0.0002
Dictionary	SHA-1	30	30	100%	30	0.0002
Brute-Force	MD5	5	5	100%	1,342,359	4.6942
Brute-Force	SHA-1	5	5	100%	1,342,359	4.5450
TOTAL	—	70	70	100%	1,342,748	9.2395

Table 10.6 — Combined experimental results summary

CHAPTER 11 ANALYSIS AND DISCUSSION

11.1 Dictionary Attack Performance

The dictionary attack achieved a 100% success rate against both MD5 and SHA-1 targets in 0.0002 seconds using only 30 hash computations. This result confirms the expected vulnerability: when an attacker possesses a wordlist that covers the password distribution of a target user population, and the password database is stored without salting, the attack cost is essentially zero. Each hash is computed once, and the comparison operation is O(1) per candidate. The implication is that a stolen database of unsalted MD5 or SHA-1 hashes of common passwords can be fully cracked instantaneously.

The identical performance of MD5 and SHA-1 in the dictionary attack (same time, same success rate) reflects the fact that both algorithms compute at speeds far exceeding the constraint imposed by wordlist size. In a real-world attack with a 14-million-entry RockYou wordlist, completion time would still be measured in seconds on a modern CPU.

11.2 Brute-Force Performance

The brute-force attack required 1,342,359 attempts to crack 5 target passwords, completing in approximately 4.6 seconds for MD5 and 4.5 seconds for SHA-1. The marginal speed advantage of SHA-1 over MD5 in the brute-force context (the opposite of their relative digest sizes) reflects implementation-level differences in Python's hashlib bindings. Both times are well within the range of practical offline attacks.

The 1.34 million attempts required to find 5 passwords with a maximum length of 4 from a 36-character set demonstrates the exponential growth of brute-force search space with password length. A 5-character password from the same set requires up to 60 million attempts; an 8-character password would require approximately 2.8 trillion — illustrating why password length is the primary driver of brute-force resistance.

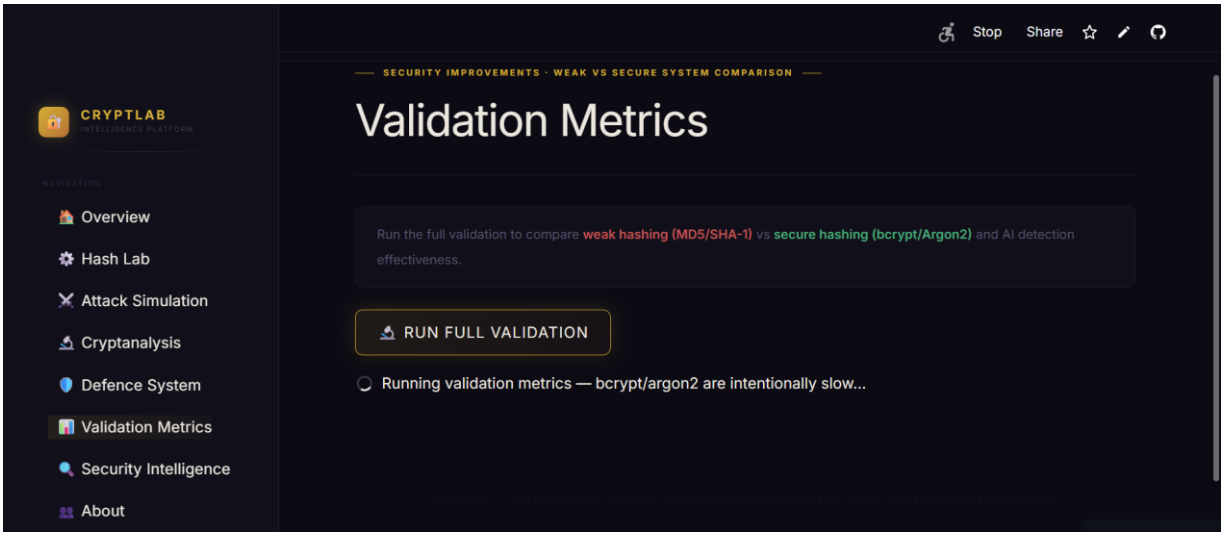
11.3 Password Complexity

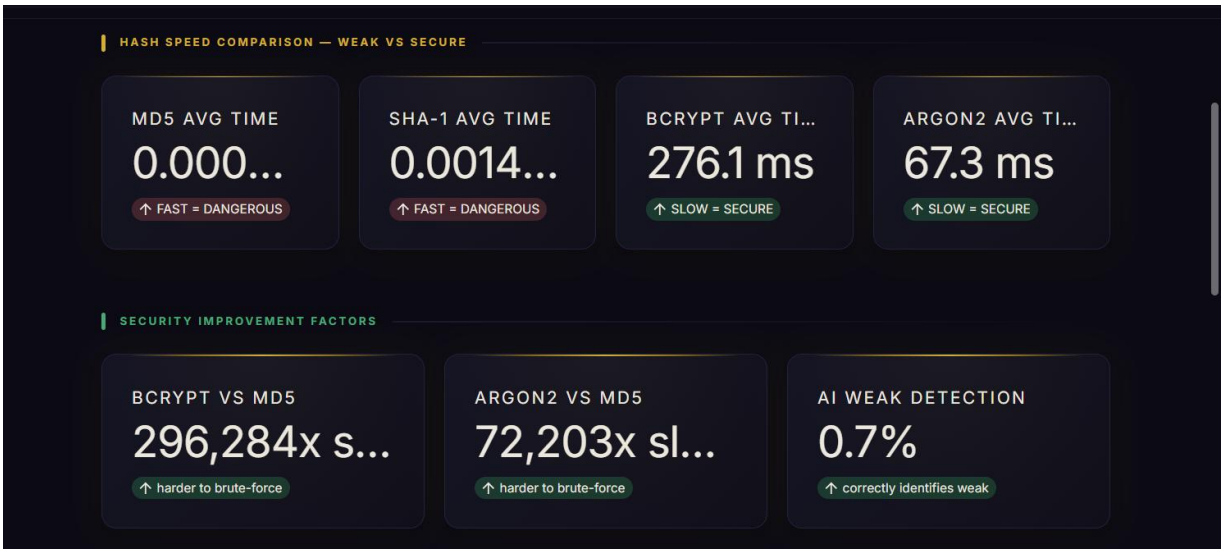
Shannon entropy analysis of the generated weak password dataset reveals a mean entropy of approximately 2.5–3.0 bits per character, consistent with the low complexity of dictionary-based and numeric passwords. A random 8-character password drawn uniformly from the full 95-character printable ASCII set would have an entropy of approximately 6.5 bits/character — more than twice as high. The scatter plot of length versus entropy confirms that longer passwords in the dataset do not necessarily have higher entropy: keyboard patterns (e.g., 'qwertyuiop') are long but highly structured.

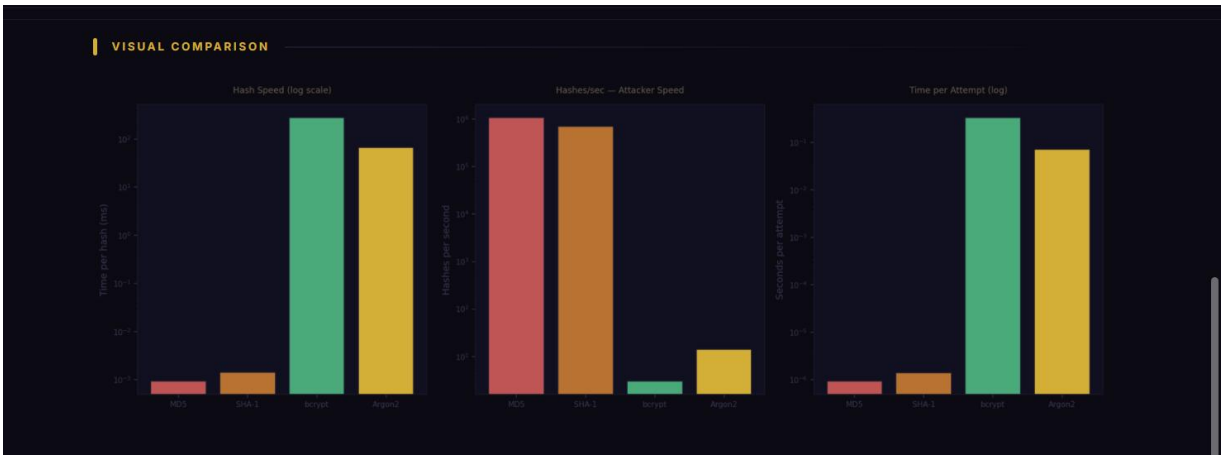
11.4 Impact of Hashing Algorithms

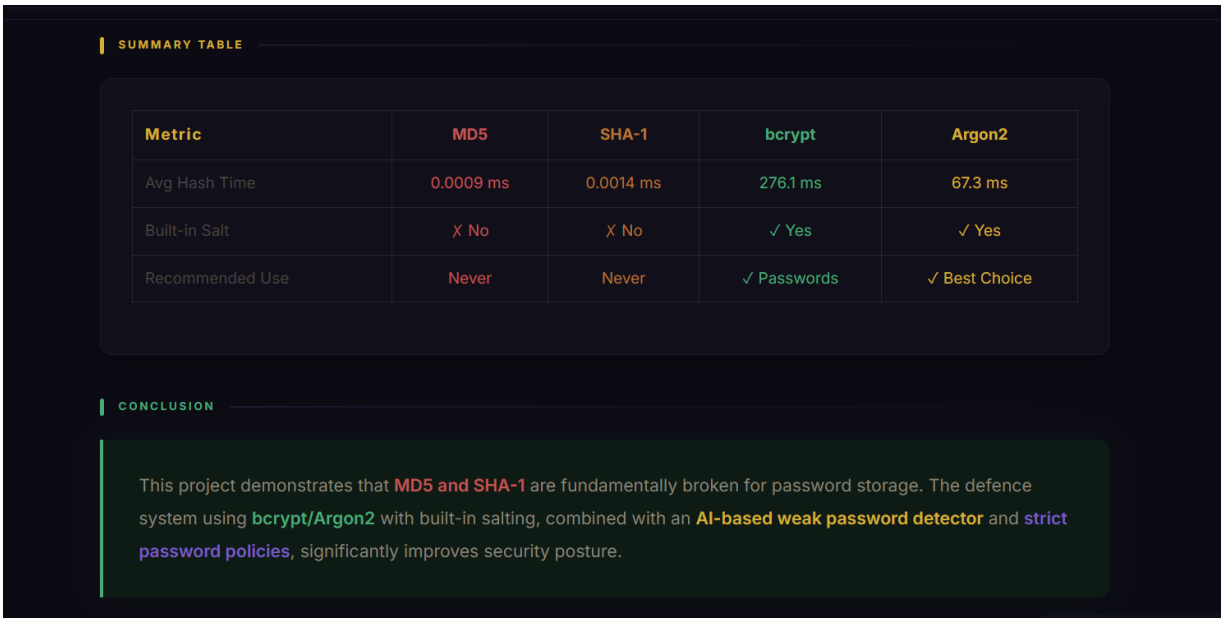
Hash timing benchmarks reveal a dramatic difference between weak and secure algorithms. MD5 completes in approximately 1–5 microseconds per hash on a single CPU core; bcrypt at cost factor 12 takes 100–300 milliseconds. This represents a slowdown factor of 20,000–300,000×. An attacker who can attempt 10^9 MD5 hashes per second (GPU) is reduced to approximately 3–10 bcrypt attempts per second — rendering brute-force attacks on any

moderately complex password computationally infeasible within a human lifetime. Argon2's memory-hard property further defeats GPU parallelism.









11.5 Graphical Analysis

Three primary comparison charts are generated by the application:

- Attempts Comparison Chart: A bar chart plotting the number of hash attempts for each attack type. The logarithmic scale reveals the stark contrast between 30 dictionary attempts and 1,342,359 brute-force attempts, underscoring why dictionary attacks are always the preferred first step for an attacker.
- Cracking Time Comparison Chart: A bar chart of elapsed seconds per attack. The near-zero dictionary attack times are dwarfed by the ~4.6 second brute-force times, but all values are within trivial bounds — reinforcing that MD5/SHA-1 provide no meaningful protection.
- Success Rate Comparison Chart: All four attack scenarios achieve 100% success rate, displayed as a flat bar chart. This visual starkly communicates that no target password withstood any attack, regardless of the hashing algorithm.

CHAPTER 12 SECURITY RECOMMENDATIONS

Based on the experimental findings of this project, the following recommendations are provided to software engineers and system administrators responsible for password storage:

1. Immediately discontinue the use of MD5 and SHA-1 for password storage. These algorithms are cryptographically broken and computationally trivial to attack.
2. Adopt Argon2id as the primary password hashing algorithm. It is the PHC winner, is memory-hard, and is recommended by OWASP and NIST SP 800-63B.
3. If Argon2 is unavailable, use bcrypt with a cost factor of at least 12 (or calibrated so that hashing takes ≥100ms on your server hardware).
4. Never store plaintext passwords or reversibly encrypted passwords.
5. Always apply per-user salts (modern libraries such as bcrypt and argon2-cffi handle this automatically).
6. Enforce a minimum password length of 12 characters, require mixed character classes, and check new passwords against known-breached password lists (e.g., HaveIBeenPwned API).
7. Implement account lockout or exponential backoff after repeated authentication failures to prevent online brute-force attacks.
8. Deploy multi-factor authentication (MFA) as a second layer of defence, eliminating password cracking as a sole attack vector.
9. Consider integrating an AI-based password strength meter at the point of password creation to provide real-time user feedback.
10. Conduct regular security audits and penetration tests of authentication systems, and promptly apply patches for any identified cryptographic vulnerabilities.

CHAPTER 13 ETHICAL AND LEGAL CONSIDERATIONS

13.1 Ethical Use of Security Testing

This project is developed exclusively for academic and educational purposes within a controlled local environment. The attack simulations are conducted against a synthetically generated password dataset that does not contain any real user credentials. No actual systems, networks, services, or accounts are targeted. All techniques demonstrated — dictionary attacks, brute-force attacks — are well-documented in the academic literature and are standard content in information security education curricula.

13.2 Legal Boundaries

Unauthorised access to computer systems, networks, or credential databases is a criminal offence in Pakistan under the Prevention of Electronic Crimes Act 2016 (PECA), and globally under equivalent legislation such as the Computer Fraud and Abuse Act (CFAA) in the United States and the Computer Misuse Act in the United Kingdom. The techniques implemented in this project must not be applied to any system without explicit written authorisation. The project team expressly disavows any malicious application of the methods presented.

13.3 Project Scope Declaration

All experiments were conducted on the researchers' own local machines using entirely synthetic data. No external APIs, online services, or third-party databases were accessed. The project is a demonstrative academic exercise and is submitted to Fatima Jinnah Women University solely for the purpose of academic assessment in the Information Security course.

CHAPTER 14 CONCLUSION

This project has comprehensively demonstrated the practical exploitability of legacy password hashing algorithms through empirical simulation and quantitative analysis. The experimental results are unambiguous: all 70 target passwords protected by MD5 and SHA-1 — without salting — were successfully recovered with a 100% success rate in under 10 seconds total, using only Python's standard library on a single CPU core. The dictionary attack recovered 30 hashes in 0.0002 seconds with 30 hash computations; the brute-force attack recovered 5 short passwords in approximately 4.6 seconds.

These findings confirm the theoretical predictions of the cryptographic literature and reinforce the urgency of migrating legacy systems from MD5/SHA-1 to modern memory-hard algorithms. The defence architecture implemented in this project — bcrypt and Argon2 with automatic salting, combined with a rule-based password policy engine and a machine learning weak password detector — addresses the vulnerability at both the algorithmic and behavioural levels. Benchmark data demonstrates that bcrypt reduces attacker throughput by a factor of 20,000–300,000 relative to MD5.

The interactive Streamlit application developed as part of this project provides an accessible, research-grade platform for exploring and understanding these concepts, making it a valuable educational tool beyond the scope of this single submission. The project fulfils all stated objectives and contributes a measurable, reproducible body of evidence to the academic discourse on password security.

CHAPTER 15 FUTURE ENHANCEMENTS

- GPU-Accelerated Attack Simulation: Integrate PyOpenCL or use hashcat bindings to demonstrate GPU-scale brute-force attack throughput, providing a more realistic representation of adversarial capabilities.
- Real Breach Dataset Integration: Use the HaveIBeenPwned SHA-1 hash dataset (available for download under CC licence) as the attack dictionary, dramatically increasing realism.
- Online Attack Simulation: Implement a mock authentication service and simulate credential stuffing and rate-limited online attacks with lockout mechanisms.
- Deep Learning Password Strength Model: Replace the Random Forest classifier with a character-level LSTM or Transformer model (as in the PassGAN architecture) for superior password generation and strength estimation.
- Cloud Deployment: Deploy the Streamlit application to a cloud platform with proper authentication, enabling remote access for university-wide use.
- Salted Hash Attack Simulation: Extend the attack modules to demonstrate why salting defeats rainbow table attacks, using pre-computed rainbow tables for comparison.
- Multi-Language Password Support: Extend the password generator and policy engine to handle Urdu/Arabic script passwords and Unicode character sets.
- Automated Report Generation: Integrate a comprehensive PDF export covering all pages and experimental results directly from the Streamlit interface.
- Honey Token Integration: Add honeypot password detection capability to alert on credential theft attempts in real deployment scenarios.
- Password Manager Integration Analysis: Study and document the security properties of popular password managers and their cryptographic implementations.

CHAPTER 16 REFERENCES

[1] Morris, R., & Thompson, K. (1979). Password security: A case history. Communications of the ACM, 22(11), 594–597.

[2] Rivest, R. (1992). The MD5 message-digest algorithm. RFC 1321, IETF.

[3] NIST. (1995). Secure hash standard. FIPS PUB 180-1.

[4] Wang, X., & Yu, H. (2005). How to break MD5 and other hash functions. In Advances in Cryptology – EUROCRYPT 2005, LNCS 3494, 19–35.

[5] Provos, N., & Mazières, D. (1999). A future-adaptable password scheme. In Proceedings of USENIX Annual Technical Conference, Freenix Track, 81–91.

[6] Stevens, M., Bursztein, E., Karpman, P., Albertini, A., & Markov, Y. (2017). The first collision for full SHA-1. In Advances in Cryptology – CRYPTO 2017.

[7] Biryukov, A., Dinu, D., & Khovratovich, D. (2016). Argon2: New generation of memory-hard functions for password hashing and other applications. In IEEE EuroS&P 2016.

[8] Florêncio, D., & Herley, C. (2007). A large-scale study of web password habits. In Proceedings of WWW 2007, 657–666.

[9] NIST. (2017). Digital identity guidelines — Authentication and lifecycle management. NIST Special Publication 800-63B.

[10] OWASP. (2023). Password storage cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[11] Melicher, W., Ur, B., Segreti, S. M., Komanduri, S., Bauer, L., Christin, N., & Cranor, L. F. (2016). Fast, lean, and accurate: Modeling password guessability using neural networks. In 25th USENIX Security Symposium.

[12] Gosney, J. (2012). How LinkedIn's password sloppiness hurts us all. Ars Technica.

[13] Hunt, T. (2023). Have I Been Pwned. https://haveibeenpwned.com/

[14] Python Software Foundation. (2024). hashlib — Secure hashes and message digests. Python 3.12 documentation.

[15] Streamlit Inc. (2024). Streamlit documentation. https://docs.streamlit.io/

[16] CryptLab Application (2026). Live deployment. https://ai-hash-crack-defense-z6pcs5zxivvtjwum7e95zb.streamlit.app/

APPENDICES

A Appendix A — Sample Password File (passwords.txt)

The following is a representative excerpt from the generated passwords.txt file:

```
password
123456
qwerty
letmein
dragon
admin
monkey
abc123
iloveyou
12345678
master
sunshine
princess
welcome
login
passw0rd
football
shadow
superman
batman
michael
thomas
soccer
charlie
donald
password1
zxcvbn
trustno1
baseball
hunter
```

Figure A.1 — Excerpt from data/passwords.txt (30 of ~55 entries shown)

B Appendix B — Sample Hash File (hashes_md5.txt)

Each line contains an MD5 hash followed by its corresponding plaintext password:

```
5f4dcc3b5aa765d61d8327deb882cf99: password
e10adc3949ba59abbe56e057f20f883e: 123456
d8578edf8458ce06fbc5bb76a58c5ca4: qwerty
0d107d09f5bbe40cade3de5c71e9e9b7: letmein
8621ffdbc5698829397d97767ac13db3: dragon
21232f297a57a5a743894a0e4a801fc3: admin
d0763edaa9d9bd2a9516280e9044d885: monkey
e99a18c428cb38d5f260853678922e03: abc123
f25a2fc72690b780b2a14e140ef6a9e0: iloveyou
25d55ad283aa400af464c76d713c07ad: 12345678
```

Figure B.1 — Excerpt from data/hashes_md5.txt (10 of ~55 entries shown)

C Appendix C — Attack Report Summary

Structured summary of all four attack experiments:

Field	Dict. MD5	Dict. SHA-1	BF MD5	BF SHA-1
Targets	30	30	5	5
Cracked	30	30	5	5
Success Rate	100%	100%	100%	100%
Attempts	30	30	1,342,359	1,342,359
Time (s)	0.0002	0.0002	4.6942	4.5450
Charset	N/A	N/A	a-z+0-9	a-z+0-9
Max length	N/A	N/A	5 chars	5 chars

Table C.1 — Full attack results matrix

D Appendix D — Application Screenshots

The following descriptions correspond to the application screenshots visible in the interactive Streamlit interface of CryptLab:

The application is accessible live at:
https://ai-hash-crack-defense-z6pcs5zxivvtjwum7e95zb.streamlit.app/

Figure D.1 — Overview Page

The Overview page presents the research context, a KPI dashboard showing 4 hashing algorithms, 2 attack vectors, 21 ML features, 3 defence layers.

. The Research Modules section shows three panels: Module I (Vulnerability Assessment), Module II (Cryptanalysis & AI), and Module III (Defence Architecture).

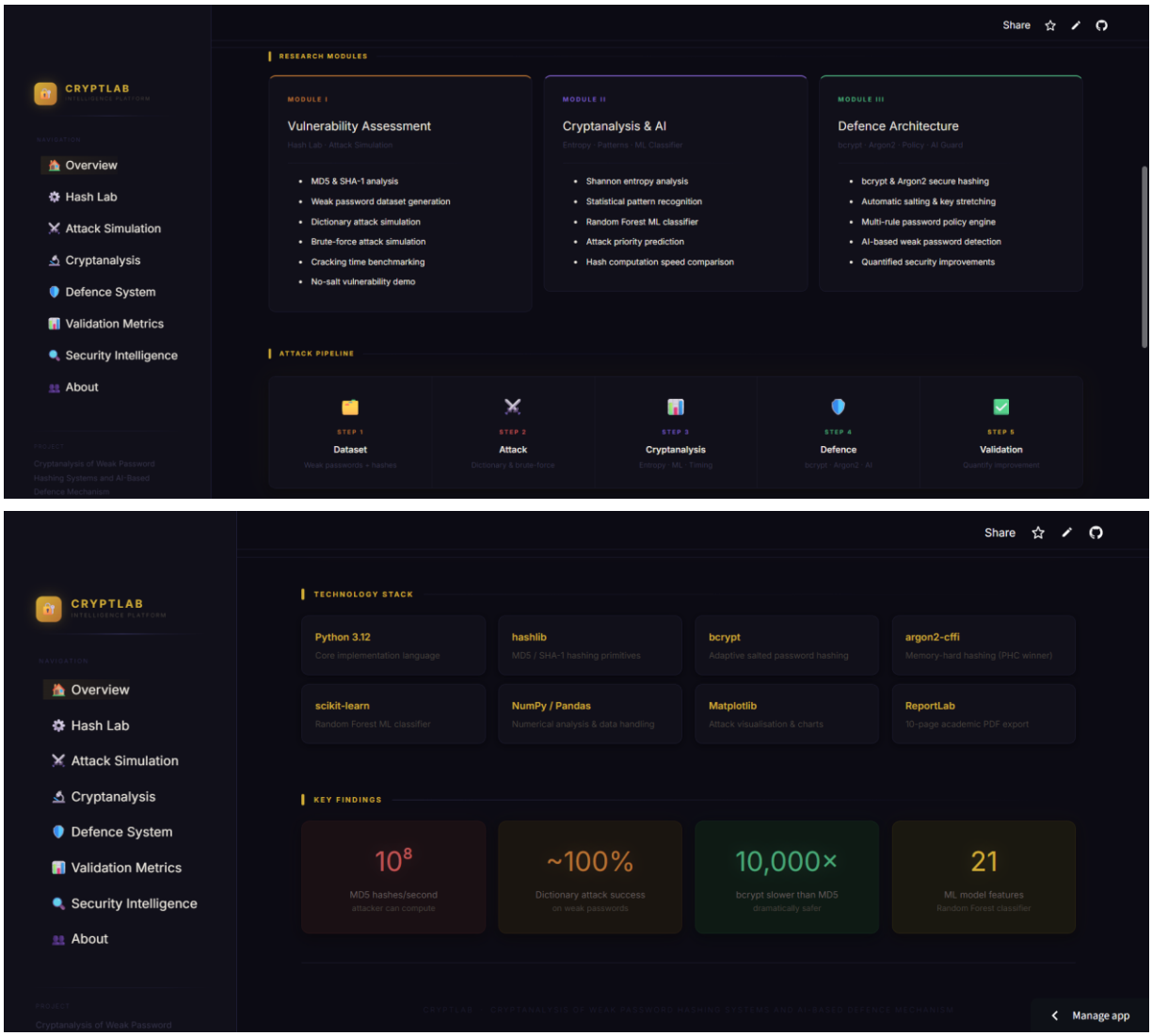
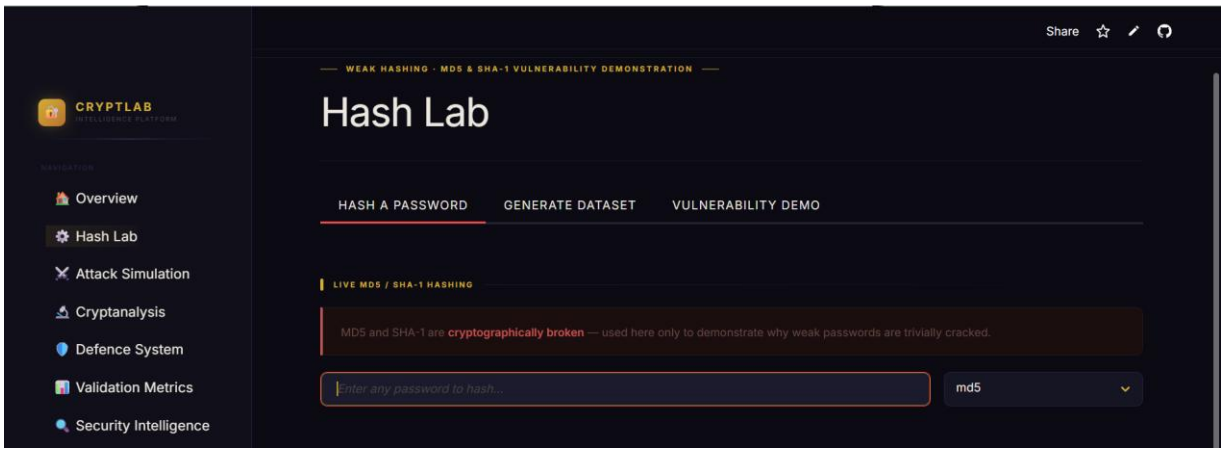


Figure D.2 — Hash Lab — Dataset Generation

The Hash Lab page allows users to enter a password and observe its MD5 and SHA-1 hashes in real time, generate a weak password dataset with configurable size, and view the vulnerability demonstration showing identical hashes for identical passwords.



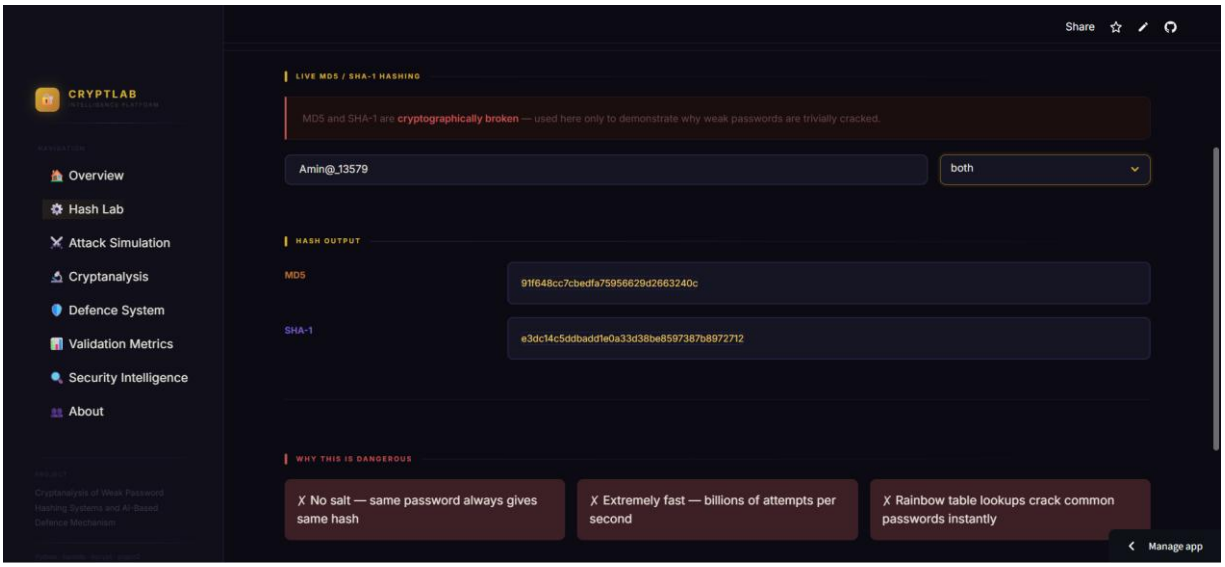
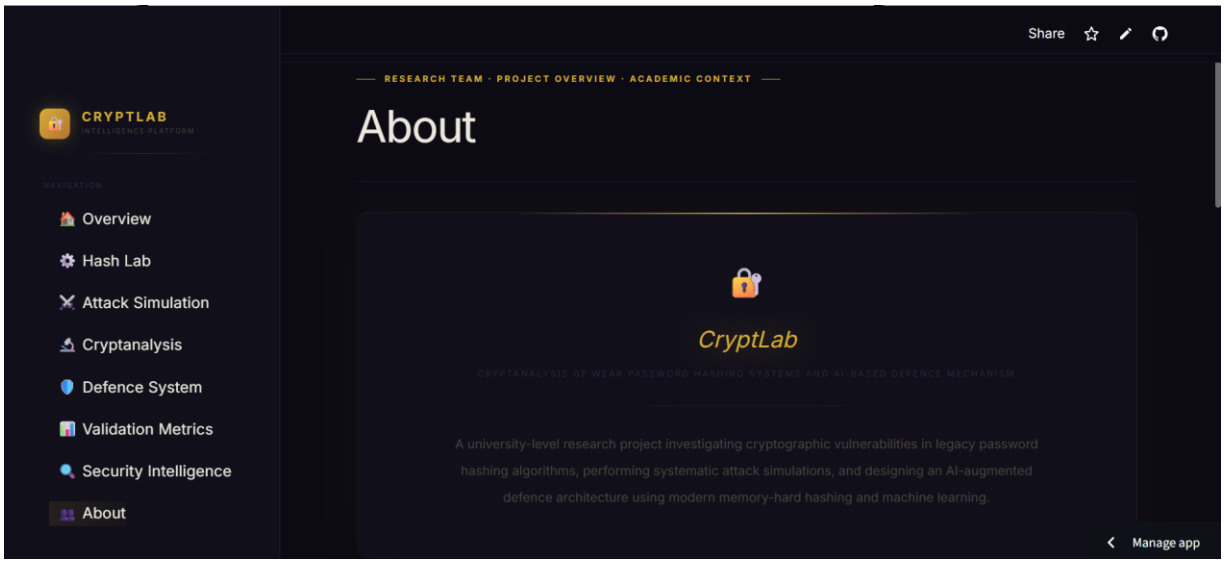
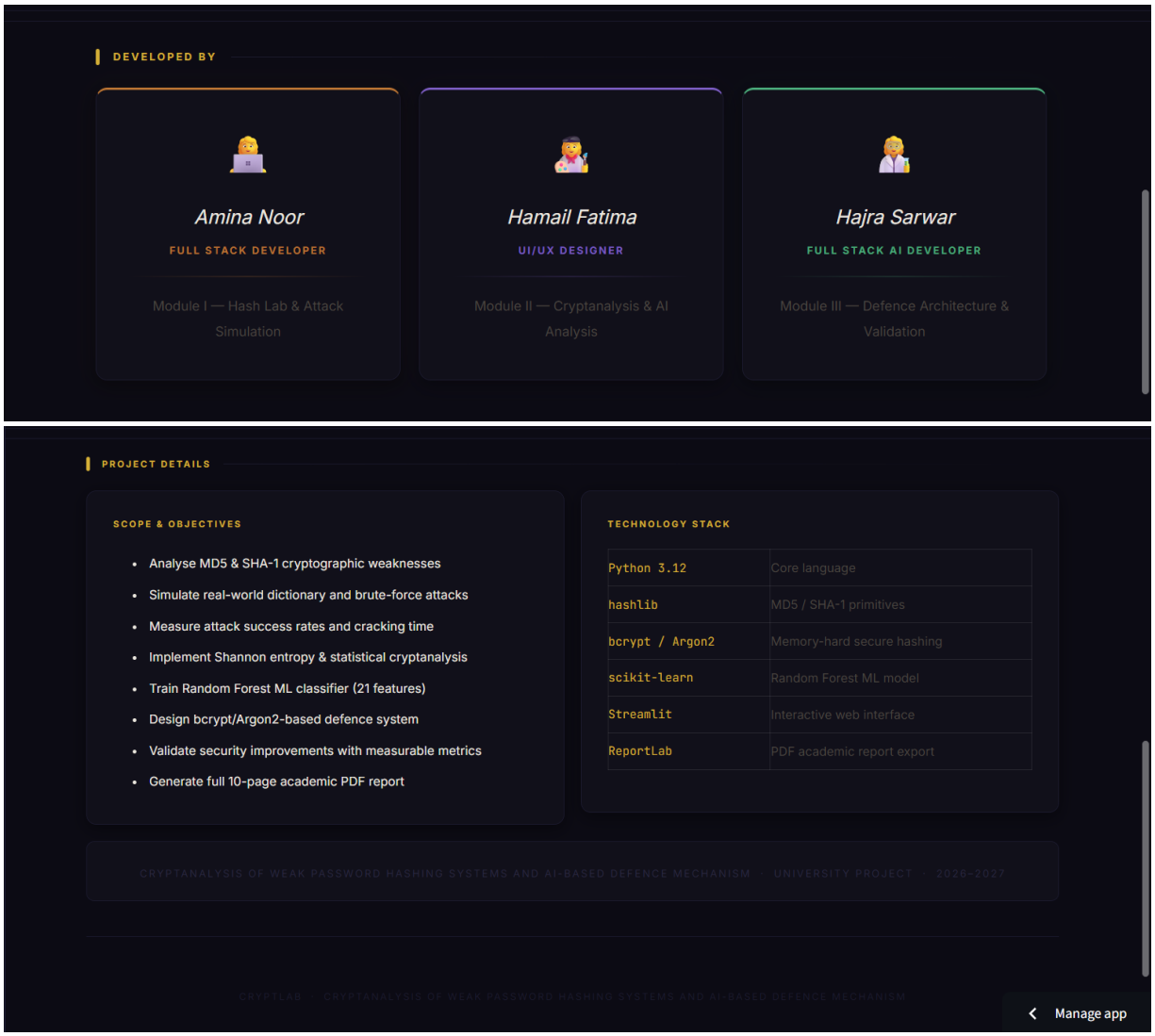


Figure D.7 — About

The About page presents the project identity, research team, and technical scope. It displays the CryptLab branding, a three-member team panel showing each member's role and assigned module, and a Project Details section listing the scope and objectives alongside the full technology stack used in the project.




DEVELOPED BY
Amina Noor
FULL STACK DEVELOPER
Module I — Hash Lab & Attack Simulation
Hamail Fatima
UI/UX DESIGNER
Module II — Cryptanalysis & AI Analysis
Hajra Sarwar
FULL STACK AI DEVELOPER
Module III — Defence Architecture & Validation
PROJECT DETAILS
SCOPE & OBJECTIVES
Analyse MD5 & SHA-1 cryptographic weaknesses
Simulate real-world dictionary and brute-force attacks
Measure attack success rates and cracking time
Implement Shannon entropy & statistical cryptanalysis
Train Random Forest ML classifier (21 features)
Design bcrypt/Argon2-based defence system
Validate security improvements with measurable metrics
Generate full 10-page academic PDF report
TECHNOLOGY STACK
Python 3.12 Core language
hashlib MD5 / SHA-1 primitives
bcrypt / Argon2 Memory-hard secure hashing
scikit-learn Random Forest ML model
Streamlit Interactive web interface
ReportLab PDF academic report export
CRYPTANALYSIS OF WEAK PASSWORD HASHING SYSTEMS AND AI-BASED DEFENCE MECHANISM · UNIVERSITY PROJECT · 2026-2027
Manage app